

A Visualized Botnet Detection System based Deep Learning for the Internet of Things Networks of Smart Cities

Vinayakumar R, Mamoun Alazab *Senior Member, IEEE*, Sriram S, Quoc-Viet Pham, Soman KP, Simran K

Abstract—Internet of things (IoT) applications for smart cities have currently become a primary target for advanced persistent threats (APT) of botnets. This paper proposes a botnet detection system based on a two-level deep learning framework for semantically discriminating botnets and legitimate behaviors at the application layer of the domain name system (DNS) services. In the first level of the framework, the similarity measures of DNS queries are estimated using siamese networks based on a predefined threshold for selecting the most frequent DNS information across Ethernet connections. In the second level of the framework, a domain generation algorithm (DGA) based on deep learning architectures is suggested for categorizing normal and abnormal domain names. The framework is highly scalable on a commodity hardware server due to its potential design of analyzing DNS data. The proposed framework was evaluated using two data sets and was compared with recent deep learning models. Various visualization methods were also employed to understand the characteristics of the data set and to visualize the embedding features. The experimental results revealed substantial improvements in terms of F1-score, speed of detection and false alarm rate.

Index Terms—Internet of Things, Smart Cities, Botnet detection, Botnets, Visualization, Deep Learning.

I. INTRODUCTION

SMART cities aim at enhancing the productivity, liveability, and sustainability of cities. Internet of things (IoT) applications for smart cities integrate information and communication technology (ICT) and different physical gadgets to enhance the efficiency of city activities and provide IoT services to residents. An IoT is a network of devices, for example, vehicles and home appliances that contain software, electronics, sensors, actuators, and connectivity which enables the things to associate, collaborate, and trade information. As indicated by a study ¹, there were nine billion smart devices in

2016, which excludes smartphones, tablets, and personal computers, and by 2020, it is foreseen to develop to 28.1 billion IoT devices. By 2025, the estimation of the IoT devices will be trillions annually [1]. Although IoT devices are placed behind firewalls or routers with network address translation (NAT), hackers could gain access to IoT devices using different advanced persistent threats (APT) due to non-standard protocols and services of IoT devices based internet protocol (IP). As individuals around the globe are progressively purchasing and installing smart devices in their homes, every one of those cheap interconnected devices causes new security problems for individuals and society in general. Indeed, even smart cities need to use new advancements to enhance energy efficiency, decrease traffic clogs and improve water quality for which they deploy more IoT devices. As the quantity of these IoT devices is incredibly growing, IoT networks of smart cities still suffer from advanced and sophisticated attacks such as botnets that are the main focus of the study.

A botnet attack refers to a group of connected computers cooperating to execute malicious and repetitive activities to corrupt and disrupt the resources of a victim such as crashing websites and domain name system (DNS) amplification attacks [2]. In IoT and smart city networks, botnets use malware to take control of IoT devices connected to web services, for example, massively exploiting IoT devices like IP-connected cameras. The botnets have currently become vast networks of multiple machines to hack a target [3]. The machines are controlled by an assaulting system named a botmaster, where each machine under the control of the botmaster is known as a bot. Criminal activity can be carried out by the botmaster by instructing each bot on the botnet at the same time to complete a coordinated task. With the increasing number of botnets over the Internet, the botmaster can cooperate to have immense repercussions both on the web and in the physical world. In this essence, large-scale activities that were previously impossible with malware could be performed by the botmaster using botnets [4]. The infected machines are under the control of the remote botmaster, who can issue attack commands on the fly to accomplish the motives.

Popular botnet scenarios have been seen, for example, the Mirai attacks that used botnets for knowing the default username and login combinations. The notorious IoT botnet brought down significant websites through gigantic distributed denial-of-service (DDoS) utilizing hundreds of thousands of compromised IoT devices. The city of Dallas was hacked on April 7, 2017, at around midnight. The emergency manage-

This work was supported by the Department of Corporate and Information Services, Northern Territory Government of Australia, the Paramount Computer Systems, and Lakshya Cyber Security Labs.

Vinayakumar R was with Division of Biomedical Informatics, Cincinnati Children's Hospital Medical Center, Cincinnati, OH, United States e-mail: (Vinayakumar.Ravi@cchmc.org, vinayakumar77@gmail.com)

Vinayakumar R, Sriram S, Soman KP and Simran K was with Center for Computational Engineering and Networking(CEN), Amrita School of Engineering, Coimbatore, Amrita Vishwa Vidyapeetham, India e-mail: (r_vinayakumar@cb.amrita.edu, sri27395ram@gmail.com).

Mamoun Alazab was with Charles Darwin University, Australia e-mail: (mamoun.alazab@cdu.edu.au).

Quoc-Viet Pham is with Research Institute of Computer, Information and Communication, Pusan National University, 46241, Republic of Korea e-mail: (vietpq90@gmail.com).

¹<http://spectrum.ieee.org/tech-talk/telecom/internet/popular-internet-of-things-forecast-of-48-billion-devices-by-2020-is-outdated>

ment system of the Dallas city was accessed by an attacker to set off 156 blaring alarms which were supposed to provide caution to the public in the occurrence of a tornado.

In the smart cities, botnet attacks control IoT devices and make these devices weaponized so that they can be used to launch DDoS attacks which can send massive amounts of bandwidth to gateways and network devices to corrupt availability of city websites [5]. To avert botnet attacks, the infrastructure of smart cities should incorporate security planning and a defense-in-depth layer. An essential control that cities can deploy network segmentation tools and different security tools such as firewalls, intrusion detection, and prevention systems, as well as access control and authentication systems. The tools could confine IoT devices from the Internet at large, which restricts the spread of an attack that misuses IoT vulnerabilities. City information technology (IT) departments ought to likewise deploy network monitoring and intrusion detection solutions that can identify when botnets are endeavoring to associate with known command and control (C2C) servers. Intrusion detection systems (IDS) can detect attacks if compromised workstations are scanning the network for finding vulnerabilities. However, the detection will likely come after a device or series of devices have already been compromised. City network managers would deploy network performance monitoring and diagnostic tools, which can give them a pattern for network traffic so they can determine whether something is peculiar.

Current cyber security tools that are available in commercial industries are mostly detecting and preventing attacks based on methods that use threshold-based, signature-based, heuristics-based, and characteristics-based on statistical measures. The methods are efficient at detecting known attacks as well as known botnet families. They entirely fail at detecting new attack variants as well as new types of botnets. These methods require domain expert knowledge to cope with the new types of botnets. The existing methods have to be continually updated manually; thus, they require more computational cost, and most of the time, there may be a high possibility that the new types of botnet could breach them and cause losses of assets for many organizations. One of the methods has used in recent days is self-learning systems that utilize machine learning algorithms [74], [75], [79], [80]. Machine learning methods are also vulnerable by adversaries that are named adversarial machine learning that should also be considered while developing robust cyber security applications based on machine learning and deep learning models [52]. Hence, machine learning models have to be continuously pre-trained to cope with the evolving nature of botnets. This involves feature engineering to identify the optimal features of botnets. On the other hand, deep learning models are capable of extracting optimal features automatically without human intervention. They can be easily trained and validated using the features of botnets. Recently, different deep learning architectures have performed well in various cyber security applications such as botnet detection, intrusion detection, malware detection systems [6], [72], [73], [78]. Additionally, the deep learning architectures have been leveraged for IoT botnet detection [3], [7].

Botmasters target various IPs to hide their mistake activity and bypass detection mechanisms. For instance, if the security device uses transmission control protocol (TCP) protocol, a hacker might change the bot protocol to user datagram protocol (UDP) one, which can easily bypass the security mechanisms. One of the common botnets targets the exploitation of the DNS protocol. This is because DNS forms an essential part of the IP, which maps domain names into IP addresses. For instance, an adversary embeds malware for a static IP or a domain name. Once the system administrator reverses engineer the malware successfully, the malware could be easily detected. However, the adversary could use fluxing methods to evade detection. There are two methodological types of fluxing: IP flux and domain flux. This paper studies the domain flux using domain generation algorithms (DGA) which create a large number of pseudo-random domain names to connect to a C2C server. Machine learning and deep learning methods have broadly leveraged for botnet detection using DNS query information [8], [9]. This study employs deep learning models for detecting botnets using DNS query information. The proposed framework is highly scalable for network DNS data analysis which is built using Hadoop on top of Apache Spark and it is used in scalable cyber threat situational awareness system [8].

The main contributions of the study are structured as follows:

- This paper proposes a botnet detection system by analyzing DNS services in the IoT of smart city applications. The framework uses DNS big data that run over high commodity hardware for accomplishing the scalability of the system.
- The proposed system includes a two-level framework for inspecting DNS logs and looking for the DGA generated domain name using deep learning architectures for reducing false alarm rates. In the first level, it uses siamese networks to find out similar domain names and passes dissimilar domain names to the second level. The second level uses a cost-sensitive deep learning architecture to effectively detect and classify the domain name into a corresponding DGA family.
- The proposed framework uses self-learning methods which can be easily trained and tested to detect APT of botnets. The developed system can be employed for effectively detecting botnet attacks in IoT networks and their smart cities.
- This paper also provides various data visualizations of well-known DNS features such as domain name length, domain name entropy and domain name n-gram using scatter plot, histogram, and density plot.

The rest of the part of this paper is organized as follows; Section II discusses the background information of DNS services. Section III provides the Literature survey of DGA and botnet detection. Section IV includes the proposed botnet detection system for IoT smart cities. Section V contains details about the preprocessing of data sets for the validation of the proposed system. Section VI provides evaluation measures. Section VII includes experiments, results, and observations.

Section VIII provided a brief discussion about the proposed work and future works. Finally, conclusion and future works are placed in Section IX.

II. BACKGROUND OF DNS SERVICES

Users use the Internet predominantly to exchange data from one gadget to either another gadget or to the servers of service providers. This data is passed over a medium through the infrastructure of an Internet service provider (ISP) to reach the destination. The IP address of the destination is required by the end-user to send the data through the Internet using DNS services. The process of the DNS query is shown in Figure 1, where the IP address utilizes the services of the DNS server by making a domain name look-up. When the request of the domain name is not locally found, the DNS server-of ISP would recursively query the authoritative DNS hierarchy.

An example of DNS operations is that when a user visits a website such as www.flipkart.com, the user types the website's name in a browser. Initially, the browser looks for the IP address of the domain name "www.flipkart.com" in its cache. If the domain name is not found, then the browser will ask the DNS servers of ISP using a DNS query. The DNS servers of ISP are generally able to provide a recursive DNS lookup which implies that the ISP's DNS server will search for the "root" name server. It does this quest to find out which name servers are authoritative for the ".com" domain. When the information related to the IP address is found, ISP's DNS server will send a DNS query to those specific name servers only. In this query, the authoritative name server for the "flipkart.com" domain is verified. After that, a final query to those specific verified name servers requesting the IP address of "www.flipkart.com" is sent. Authoritative DNS servers are those servers that acknowledge and reply to www.flipkart.com that can also solve this IP address problem. These authoritative DNS servers could be the company's private DNS server, or it might also be the company's hired DNS service provider which manages the DNS service.

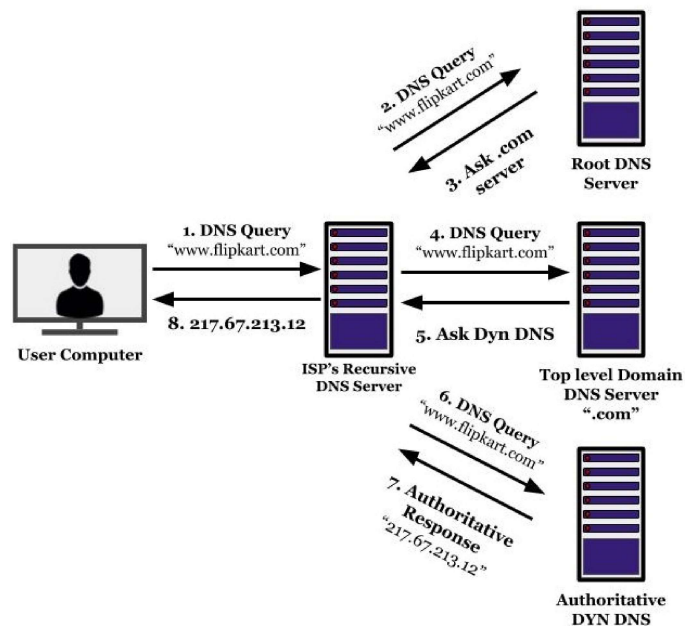


Fig. 1. Steps of DNS query request-response system.

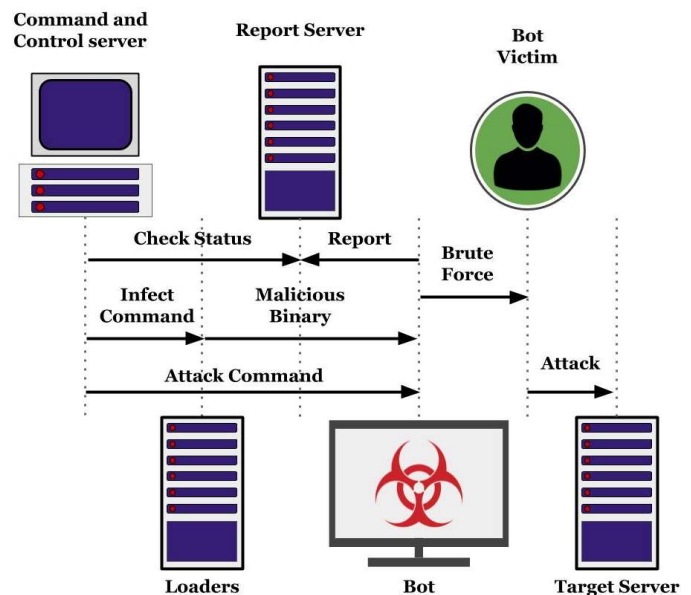


Fig. 2. Steps of operating Mirai botnet.

A. Background knowledge on IoT botnet triage

LightAidra/Aidra, BASHLITE, and Mirai are well known IoT botnet malware. The source code of the malware is available online and is mostly written in 'C' and Google 'Go' language. There are key components in IoT botnets, as explained below.

- Bots are the compromised IoT devices that perform DDoS attacks on order.
- C2C servers are employed by the botmaster to control the bots.
- Reporting servers are used to gather the outcomes from bots (or other external scanners).
- Scanners are utilized to scan vulnerable IoT devices.
- Loaders sign onto vulnerable IoT devices and thereby train them to download malware.
- Malware distribution server is where the malware code is stored. Infected IoT devices download the malware codes from this server.

Other components can execute the functionalities of IoT botnets. For example, in the Mirai attacks, there is no separate scanner component; therefore, bots perform the function of scanning, as depicted in Figure 2.

The essential operations of most IoT botnets are as follows. Initially, malware, bots, external scanner or C2C servers regularly examines the web for vulnerable IoT devices. Once a vulnerable IoT device is discovered, utilizing a list of known default credentials along with brute force, malware or bots or external scanner or C2C servers access the vulnerable IoT devices. At that point, the reporting server stores the device IP address along with the successful password to be used by malware later. To download additional malware payload, it

TABLE I
SUMMARY OF BOTNET ATTACKS USING DDoS AS A SERVICE.

Malware Name	Description	Year
Mirai	In recent, the most common botnets used for DDoS is Mirai, which targets mostly CCTV cameras, DVRs, and home routers. It infected around 4,000 IoT devices within an hour, and around half a million devices were infected. The source code was publically made available, and it has led to increasing the number of IoT infected devices from 213,000 to 483,000 in two weeks [10], [11].	2016
KrebsOnSecurity.com	The Brain Krebs blog experienced a 623 Gbps DDoS attack. The security researcher blog was attacked using a massive number of compromised IoT devices [13]. Mirai and BASHLITE attack infected these IoT devices. This DDoS attack was viewed as a striking back activity from a group of hackers because of Krebs series of articles on vDOS [14]. Akamai (a leading CDN operator) platform could not handle this aggressive DDoS attack [15] and after three days terminated the pro bonus contract. Five different DDoS attacks ranging from 123 Gbps to 623 Gbps was done on the Krebs blog [12].	2016
OVH	A series of DDoS attacks, with many ranging larger than 100Gbps, was experienced by OVH. This French cloud computing and hosting company also experienced a severest single attack which reached 799 Gbps [16]. The most massive DDoS attack launched so far, where it was recorded when 145,607 cameras or DVR generated at least 1.1 to 1.5 Tbps traffic. These compromised IoT devices sent traffic between 1-30 Mbps and were infected by BASHLITE and Mirai malware.	2016
Dyn	Dyn managed DNS servers experienced a DDoS attack from 100,000 compromised IoT devices. The leading managed DNS provider in the world was attacked IoT devices infecting them by the Mirai malware [17], [18].	2016
Deutsche Telekom	Deutsche Telekom experienced a connectivity problem, which was a new Mirai malware variant. This Mirai malware scanned and infected vulnerable devices. D1000 modem manufactured by Zyxel did not correctly implement TR069/TR064 [19].	2016
CCTV vs Small Business Websites	A DDoS attack exploited thousands of websites like small jewelry shop websites. This attack lasted for days and was from more than 25,000 compromised CCTV devices. 5% of the IPs observed in this attack were IPv6 [20].	2016
Liberia	Mirai Attacks Twitter account observed the Mirai attack. Mirai might have brought down the Internet in Liberia was speculated in [21], where the authors in [22] claimed that the failure of Internet access in Liberia was due to Internet undersea cable.	-
Lappeenranta	In Lappeenranta, two housing blocks experienced disruption of heating distribution, which was due to DDoS attack by the Mirai botnet [23], [24]. The system rebooted the central control to mitigate the attack but got caught in an infinite restart loop. For more than a week, the heating system was offline due to this infinite restart loop.	2016
Russian Banks	At least 5 Russian banks experienced a DDoS attack. Using this attack, 24,000 compromised IoT devices were utilized which were distributed around 30 countries [25], [26], [27].	2015
US Elections	DDoS attacks were observed on the campaign website of Donald Trump and Hillary Clinton [28]. On 6th November 2016, the DDoS attack lasted for 24 hours on TCN, which was used by-election campaigns [29]. This attack started by sending a small flood of junk traffic but increased progressively until all the connections got saturated.	2016
WikiLeaks	A DDoS attack, which was nearly for 24 hours, made the email publication servers of Wikileaks to be offline. This DDoS attack was in response to "DNSLeak2" [30], [31].	2016
Random DoS	In Random denial of service (RDoS), Hackers sent ransom letters to organizations to inform them they were exploited. Many cyber criminals utilized DDoS as a Service to send out some warning shots and are reported to earn \$100,000 [32].	-

exploits the known security weaknesses of the IoT devices. When the execution of the downloaded malware binary is completed, the bot malware becomes active. The botmaster does not execute other malware to access the same IoT device. If other malware is found in the IoT device, diverse techniques like memory scraping are used to eradicate them by a bot that reconfigures the device to be a piece of the botnet. To indicate their existence to the botmaster, communications between bots and their C2C servers happen regularly using an internet relay chat (IRC) based protocol. Until the botmaster instructs bots to perform a DDoS attack, the malware in bots stay lethargic and does not affect the IoT device performance. Table I summarizes the most common botnets, including their DDoS attacks as services.

III. RELATED STUDIES OF DGA AND BOTNET DETECTION

This section discusses the related studies of botnet detection systems using DNS queries. It also explains a summary of their detection methods that are relevant to the proposed botnet detection system, including domain name similarity checker, DGAs, and botnet detection based deep learning.

A. Related works for string similarity detection

Homoglyph or name spoofing attacks are used to obfuscate domain names, and their existing methods rely on string matching and Levenshtein or edit distance functions, which are computationally heavy and produce high false positive results like shown in the Figure 3. In the figure, an example case of domain misclassification is shown. the method based on edit

distance would classify a legitimate domain as a spoof and a spoofed domain as real.

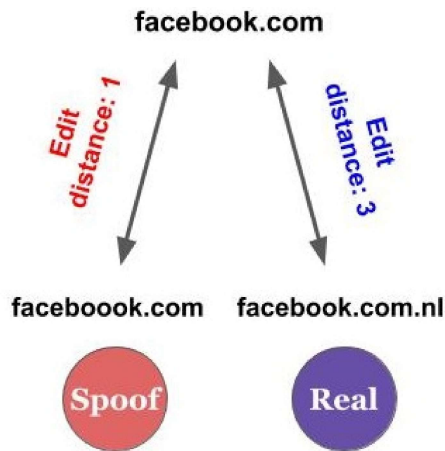


Fig. 3. An example of Misclassification by edit distance based classifier.

Several studies have conducted for applying string matching techniques. For example, in [33], [34], the authors focused on developing efficient string matching methods. Likewise, in [35], the study concentrated on enhancing the quality of nearest neighbor search models. However, those methods are not effective for detecting name spoofing due to their high computational resources [36]. Consequently, multiple studies proposed name spoofing detection techniques based on visual comparisons. In [37] [38], the authors developed custom edit distance functions, which are dependent on the visual similarity of characters. Replacement of character by a visually similar character could result in a smaller edit distance than a visually different character. The techniques depend on manually derived similarity measures between the characters, and they did not include a sizeable Unicode set. These techniques were compared with the standard edit distance method in [36], and they still produce a high false positive rate.

Siamese neural networks techniques were introduced by LeCun and Bromley in 1993 to validate handwritten signatures [39]. They are class neural network architectures that contain two or more neural networks, along with utilizing the same configuration, parameters, and weights. These networks take distinct inputs and a simple comparative energy function (Euclidean distance) is used to compare their outputs. They are usually used to find similarity or relationship between two or more comparable things. In [36], the author proposed a metric-learning technique based on a siamese convolutional neural network (S-CNN) model for detecting domain and process homoglyph attacks. The proposed model was trained on the strings rendered as images for detecting the visual similarity of the strings. Following [36], the performance analysis of various deep learning based siamese architectures are evaluated for detecting domain and process homoglyph attacks [77]. The detailed summary domain name homoglyph attacks is tabulated in Table II.

B. Related works for botnet and DGA detection based deep learning

Several studies have conducted for identifying botnets and DGA using deep learning models. For instance, Woodbridge et al. [40] presented a feature-less real-time DGA classifier, which uses a long short-term memory (LSTM) network. The performance of the classifier outperformed the random forest (RF) model using manually-crafted features [4]. In [48], the authors discussed various issues related to synthetic data, explicit labeling and proposed a deep learning approach for DGA detection to overcome the issues. They trained LSTM, convolutional neural network (CNN), and RF with a large volume of real-time data and presented a detailed comparative analysis. Yu et al. [44] compared the performance of a recurrent neural network (RNN) and CNN deep learning models, which perform a character-based text classification for detecting DGAs. The outputs of the models are almost the same in terms of accuracy and false positive rates.

Lison et al. [41] proposed an RNN deep learning model for detecting malicious domain names generated by DGAs. The model was trained on a large data set, which contains samples with a total of 61 malware families. In [42], the authors studied the performance of deep learning architectures such as RNN, Identity RNN (IRNN), LSTM, CNN, and CNN-LSTM for detecting malicious domains generated by DGAs. They used a combination of publicly available data sets and real-time data of OSINT DGA. In [9], the authors discussed various deep machine/deep learning methods to analyze large volumes of DNS logs to identify DGA based malware and block the communication channels between the botnet and C2C servers. They trained LSTM, RNN, RF, decision tree (DT), and Naive Bayes (NB) models using two different data sets and concluded that deep learning approaches perform better than classical machine learning models, and LSTM obtained high malicious detection rates compared with other deep learning models.

In [8], the authors proposed a scalable framework which, collects DNS traffic data at the ISP level and detects DGA based malware in near-realtime. They employed deep learning approaches such as LSTM, gated recurrent unit (GRU), RNN, IRNN, CNN, and CNN-LSTM for malware detection compared to classical machine learning methods. Mohan et al. [45] proposed a new deep learning architecture for detecting malignant URLs and malicious domain names generated by DGAs. In [46], the authors compared the performance of architecture proposed in [45] with other deep learning architectures on a large-scale data set. The experiments revealed that the architecture outperformed other deep learning approaches while monitoring DNS traffic. Curtin et al. [47] proposed a new RNN based approach for classifying DGA generated domain names. The approach used additional information, such as WHOIS for increased performance. In [43], a novel method was suggested for DGA botnet detection. The method overcomes the multiclass imbalance issue in the cost-sensitive LSTM by integrating a binary and multiclass cost-sensitive LSTM model. In [58], the importance of time-split is discussed over random-split for DGA detection and malicious

TABLE II
SUMMARY OF DOMAIN NAME HOMOGLYPH ATTACKS.

Reference	Method	Summary
[38]	Edit distance	• Uses string comparison algorithm, edit distance (Levenshtein) to measure the number of edits (insertions, deletions, substitutions or transpositions) to convert one domain name to another. • This approach is computationally expensive and produces a high number of false positives.
[37]	Visual edit distance	• Uses an enhancement of [38].
[36]	Siamese network	• Uses siamese network based on CNN to learn the similarity of pair of domain names. • This method is computationally inexpensive when compared to [38], [37] and the performance is better than the edit distance-based methods. • The features from domain names are extracted implicitly, so more robust in an adversarial environment when compared to classical edit distance based methods. • This approach is sensitive to the imbalanced pair of domain name data set.
[77]	Siamese network	• Evaluates the performance analysis of various deep learning architectures with siamese networks to detect domain name spoofing attacks. • This method uses state of the art approach of NLP, embedding to transform the domain name into vector representation instead of transforming the domain name character into images. • This method can perform well compared to [36] with a large number of Unicode data set. However, this approach is sensitive to the imbalanced pair of domain name data set.
proposed	Siamese network	• Proposes cost-sensitive deep learning architectures with siamese network to detect domain name spoofing attacks.

uniform resource locator (URL) detection using deep learning architectures.

Feng et al. [49] studied the performance of deep learning ImageNet models for the classification of domain names generated by DGA. They presented a comparative study of pre-trained deep learning, such as AlexNet, VGG, SqueezeNet, Inception, and ResNet in the context of accuracy, memory, and speed. Mac et al. [51] studied the performance of various supervised learning methods such as Hidden Markov Model (HMM), DT, Support Vector Machine (SVM), recurrent SVM, LSTM, CNN-LSTM, bidirectional LSTM, and extreme learning machine for DGA analysis. The results showed that recurrent SVM and bidirectional LSTM performs better with high detection rates for both binary and multiclass classification. In [50], a detailed analysis of various feature reduction methods, classical machine learning, and deep learning methods was studied for zero-day botnet identification. Anderson et al. [52] used generative adversarial networks (GAN) to generate adversarial domain names randomly and successfully bypassed the DGA classifiers.

Two-shared tasks on the detection of DGA generated domain names were suggested in [53]. The first task classified domain names as either legitimate or DGA generated, while the second subtask discovered the DGA generated domain names to its corresponding DGA malware families for improving the accuracy detection. Although deep learning methods improved detection accuracy to some extent, they demand further analysis to filter DNS features while detecting botnets, enhance scalability and reduce false positive rate, as we propose in this study. Table III summarizes various DGA detection methods.

IV. PROPOSED BOTNET DETECTION SYSTEM FOR IOT SMART CITIES

The proposed botnet detection system was designed using a two-level framework for semantically determining DNS features, as shown in Figure 4. The framework intercepts DNS packets and employs preprocessing to convert the packets into human-readable format particularly DNS logs inside an Ethernet local area network (LAN). The domain names are extracted and visualized from DNS logs. After the visualization, the domain names are passed into a similarity checker model. This loads the trained siamese network models and checks for the similarity-based on estimating a score for the domain name features. The estimated score is compared with a predefined threshold, and if the score is lower than the threshold, then the domain name is flagged as spoof otherwise passed into the DGA classifier for further analysis. The classifier uses a cost-sensitive model-based deep learning, determines DGA generated domain, and categorizes the DGA domain name into the corresponding DGA categories. Finally, the results are passed into a Front End Broker module, which displays the detailed results to a network administrator. The modules of the proposed botnet detection system are discussed below.

A. Module of data visualization

- **A scatter plot-** is a visual representation that can be used to represent the relationship between two variables. The first variable is plotted on the X -axis, and the second variable is plotted on the Y -axis. It is also called a correlation plot because the plot gives the correlation between the two variables.
- **A histogram plot-** provides a graphical representation of the distribution of data using statistical information. A

TABLE III
SUMMARY OF DGA DETECTION METHODS.

References	Method	Summary
[54]	Blacklisting	Blacklisting is a good method to detect known DGA patterns, but it completely fails to detect the variants and new DGA generated domains.
[4], [55], [56]	Retrospective method	- The retrospective methods are based on placing similar domains into groups and using their statistical properties to determine if they are DGA generated. These methods cannot be used in realtime detection and prevention systems because they are time-consuming processes. - Also, many of these techniques, use contextual information such as passive DNS and aggregations of all NXDomains throughout the network. Such requirements are not only costly to integrate, but they also may not be possible due to real-world constraints of many systems (such as endpoint detection). - The model proposed detects DGA in real-time based on the domain names.
[40]	Real-time detection	The proposed model is sensitive to class imbalance, which limits its ability to detect families with very little support from the training set.
[57]	Real-time detection	- It is a feature-based approach that relies on additional NXDomains information to facilitate DGA detection. - The model is sensitive to imbalanced data.
[58]	Real-time detection	- It proposes a featureless model to detect DGA domains. The train and test data sets are completely disjoint and collected in different timings. These data sets have varying patterns. Thus, the models that can perform well on this data, can be more generalizable and deployed in real-time systems. - The proposed model is sensitive to imbalanced data.
[49], [50], [51], [47], [45], [46], [8], [9], [42], [41], [44], [48], [53]	Real-time detection	- These works have analyzed the performance of various Deep Learning models. - They are sensitive to imbalance data.
[43]	Real-time detection	- An LSTM based framework is proposed for real-time detection of DGA. The model handles imbalanced data. However, most of the experimental analysis is on only one data set. - There is no analysis of the time-based data splitting, as well as the data set is from a single source.
proposed	Real-time detection	- It consists of a two-level framework to detect DGA domains effectively. The first level can detect the spoofed domains and the second level, further, analyzes the domains to detects DGA generated domains. It also categorizes the DGA generated domains into corresponding categories. - The proposed approach handles the imbalanced data problem.

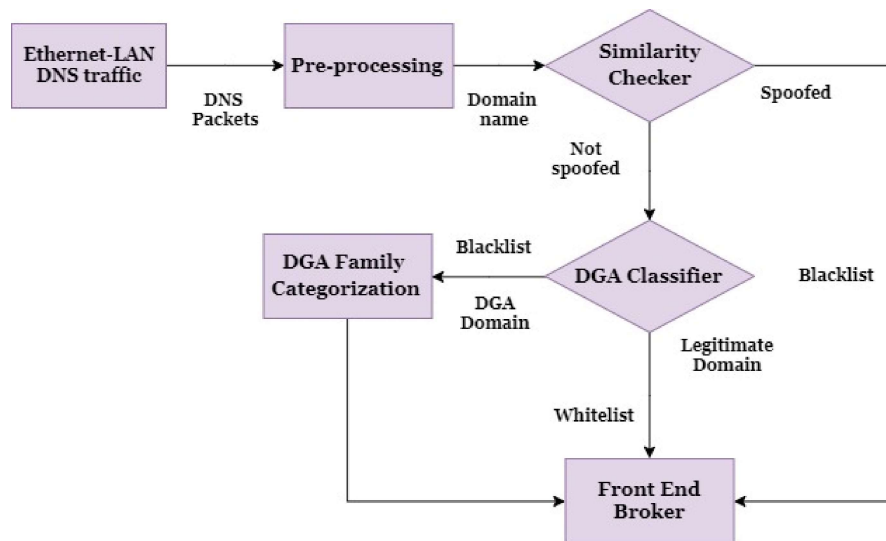


Fig. 4. Proposed botnet detection system.

mostly independent variable is used along the X -axis, and the dependent variable is used along the Y -axis. Finding an appropriate bin in a histogram is very important because it can change the shape of the distribution. This can be avoided using a density plot.

- **A density plot**- is a smoothed form of the histogram since it represents the relationship between different variables based on estimating their probability density function.

B. Importance of cost parameter in deep learning architectures

The cost-sensitive deep learning architectures consider all the data samples equally. Thus these types of models are naturally quite sensitive to the class imbalance problem. These models give importance to the classes that have more number of samples during training and limit the learning capability to the classes that have very less number of samples [43]. Cost-sensitive learning served as an important method in real-world data mining applications and provides an approach to carefully handle the class imbalance problem.

The present work follows the [43] where they followed the algorithmic approach to directly include the misclassification costs into the deep learning architectures backward pass. Each sample S is interconnected with a cost item $C[class(s), p]$, where $class(s)$ and p are the actual and predicted class respectively. The class item represents the classification importance which provides lower value for the classes that has large number of samples and higher value for the classes that has smaller number of samples. However, the cost matrix is unknown at the beginning stage. To identify the optimal cost matrix, genetic algorithm can be employed. This method consumes more time and it is a tedious task in practice [43]. Let's assume that the samples have equal cost at least in one category. $C[i, i]$ indicates the misclassification cost of the class i , which is generated using the class distribution as

$$C[i, i] = \left(\frac{1}{n_i} \right)^\gamma \quad (1)$$

Where $\gamma \in [0, 1]$. 0 indicates that cost-sensitive deep learning architectures are diminished to cost-insensitive and 1 indicates that $C[i, i]$ is inversely proportional to the class size n_j .

C. Methodology followed to convert the domain name into vector representation

This work leverages Keras embedding² layer to convert the domain name into its vector representation. The Keras embedding layer takes 3 parameters; dictionary size, embedding size and the length of the input sequence. These 3 parameters are hyperparameters and can have an impact on the performance. Initially, the Keras embedding initialized with random weights. These weights are tuned during backpropagation with respect to other deep learning layers. The process involved in the mapping Keras embedding to domain name representation is shown in Figure 5. Initially, we extract the domain names

from the DNS query and employ preprocessing. Preprocessing involves extraction of second-level domain name and conversion of characters to small letters. Differentiating between the small and capital letter may result in regularization issues and sometimes require more number of parameters. Then we create a dictionary and an empty place with unknown is added to handle the new character during the testing phase. Then all the characters are replaced by the corresponding index value in the dictionary. These domain name numeric vector are of different shape. The maximum length of the domain name can be taken as the maximum length of the domain name input sequence or the concepts of statistics can be employed to identify the optimal maximum length. Then the domain name vectors less than the maximum length are padded with 0s.

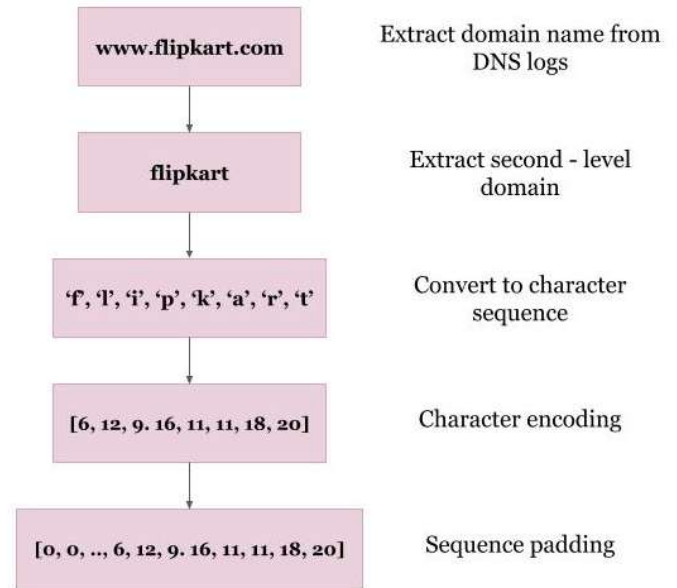


Fig. 5. Domain name vector representation using Keras embedding.

D. Similarity checker module

The primary objective of this study is to find out whether the set of domain names is similar or not. It can be modeled as a binary classification problem. Let us consider m number of a pair of domain names:

$$D = \{(d_{11}, d'_{12}, y_1), (d_{21}, d'_{22}, y_2), \dots (d_{m1}, d'_{m2}, y_m)\} \quad (2)$$

Where d_{m1} denotes that the domain names are not spoofed, d'_{m2} denotes that the domain names are spoofed and in $y \in \{0, 1\}$, 0 indicates spoofing and 1 indicates no spoofing.

There are two key steps of identifying the domain names that are similar or not. The first step selects an appropriate feature representation and the second step performs a prediction function. The feature representation constructs the n -dimensional vector representation m_n which can be passed into a prediction function as input $y_n = \text{sigmoid}(f(m_n))$. The main aim here is to minimize the total number of misclassifications by minimizing the loss function. The loss function can also include a regularization term.

²<https://keras.io/layers/embeddings/>

A siamese network method is used to identify the visual similarity between the domain names and the whitelist of possible domain names that an attacker may spoof. It takes a pair of strings such as domain or process names with a score '0' for similar and '1' for dissimilar. The proposed method is shown in Figure 6, which represents the following steps to identify whether the domain or process name is spoofed or not-spoofed. First, a preprocessing unit is employed for a set of domain or process names that includes conversion of all characters to small letters to avoid the regularization issue. Here, the Keras character embedding layer is used as the initial layer. A dictionary creation is used for transforming a domain name into a vectorized domain name. A valid list of characters for a domain name and process name dictionary is given below.

Valid character set for a domain name: -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z

Valid character set for process name: x01, space, !, #, \$, &, ' (,), +, ,, -, ., 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, =, ?, @, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, [,], a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z

Second, the optimal feature vectors are estimated, and a similarity score is also estimated based on comparative energy function (Euclidean distance function). Euclidean distance function with two distinct neural networks can be mathematically defined as follows:

$$d_W(m_1, m_2) = E(NN_W(m_1), NN_W(m_2)) \\ = ||NN_W(m_1) - NN_W(m_2)||_2 \quad (3)$$

Where (m_1, m_2) denotes a pair of a domain name, NN_W denote a neural network function parameterized by W . The proposed similarity checker architectures flag the pair of a domain name as similar if the $d_W(m_1, m_2)$ is small, otherwise dissimilar. If the distance is less than a predefined threshold, it is assumed to be a spoof. The empirical discussions of the similarity checker module and its predefined threshold values are explained in Section VII-B. To minimize w, the contrastive loss [43] is employed.

The proposed method replaces the computer vision technique [36] using the natural language processing approach as the domain names are simply strings. The computer vision technique relying on the conversion of characters into image formats, but the proposed method directly uses the character level embedding approach, which helps to learn the sequence and semantic information of characters. Moreover, the proposed method is computationally inexpensive. In addition, the proposed methodology is insensitive to imbalanced set of domain name data.

E. Module of DGA detection based deep learning

The main target of this module is to classify a given domain name as either a legitimate pattern or a DGA generated like shown in the Figure 7 and to categorize the DGA domains into their corresponding category. The classification problem is binary when the domain name is classified into either DGA

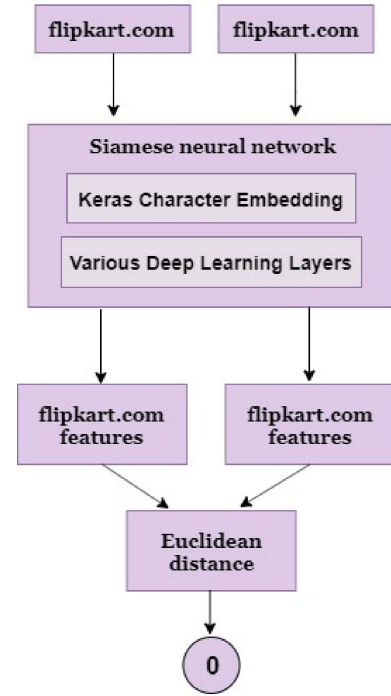


Fig. 6. Domain name Similarity checker.

generated or legitimate, while the classification problem is multiclass when the domain name is classified into legitimate and categorizations of the DGA generated domain name into the corresponding DGA families. Let us consider a set of domain names $D = \{(d_1, y_1), (d_2, y_2), \dots (d_n, y_n)\}$, where d_i represents a domain name and y represents '0' for legitimate and '1' for DGA generated in the binary classification, and in the multiclass classification, the y value depends on the number of DGA malware families.

The proposed method takes a domain name as an input which is composed of a sequence of characters as inputs and transforms it into a sequence of vectors, as employed in the Keras character level embedding. After converting characters into a vector, the entire vectors are passed into deep learning layers such as various types of recurrent layer, convolutional layer, and a combination of recurrent and convolutional layers. These layers process the input vectors one by one and update a hidden state vector at each step.

We employ a cost-sensitive and cost-insensitive deep learning model in the DGA detection and categorization, which is depicted in Figure 8. The diagram located on top shows the training process of the cost-insensitive deep learning model, while the other diagram presents the training process of the cost-sensitive deep learning model. In the cost-sensitive deep learning model, cost-weights are used to give importance to the DGA family, which has a very less number of domain names and to give less importance to the DGA family, which has more number of samples. This enables us to avoid the imbalanced problems in the multiclass DGA classification. Both classifiers include a character level embedding layer, a deep learning layer with cost-sensitive and without cost-sensitive, and a dropout and fully-connected layer.

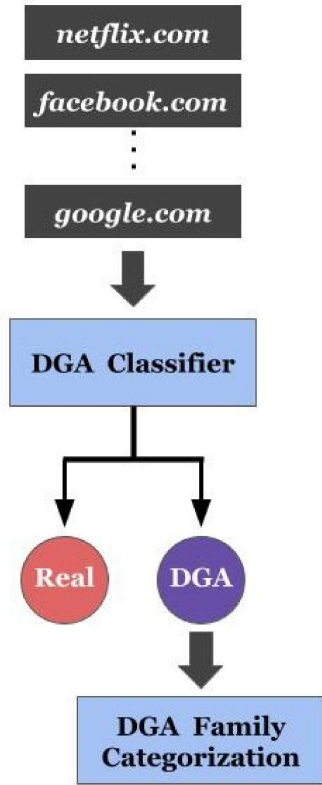


Fig. 7. An example of our proposed DGA classifier.

The pseudocode for the proposed method is given below; Vectorized Domain $\rightarrow$ Keras character level embedding $\rightarrow$ RNN, LSTM, GRU, IRNN, B-RNN, B-LSTM, B-GRU, B-IRNN, CNN, CNN-RNN, CNN-LSTM, CNN-GRU $\rightarrow$ Dropout $\rightarrow$ Dense $\rightarrow$ *Sigmoid* for binary and *Softmax* for multiclass.

We used benchmark character-based models for DGA detection and classification. The short description of these models is listed in Table IV. The workflow of the proposed model for DGA detection and classification as follows:

- 1) Keras character level embedding layer: transforms the domain name vector representation into a dense vector representation. Characters in a domain name are closer to each other in a vector space.
- 2) RNN, LSTM, GRU, IRNN, B-RNN, B-LSTM, B-GRU, B-IRNN: allows the model to learn important features (patterns of characters) from domain names and capture dependencies between non-adjacent characters.
- 3) Convolution layer: filters the embedded inputs to form an activation map, which represents the locations of discovered features in the data embedding
- 4) Pooling layer: controls the spatial size of the representation to decrease the number of parameters and memory footprint in a network for overcoming the overfitting problem.
- 5) Dropout layer: prevents the model overfitting problem by setting a random subset of neurons to zero.
- 6) *Sigmoid* and *Softmax* activation functions: are classical transformation functions to assign a probability

that a domain is malicious. *Sigmoid* outputs are 0 or 1, where 0 indicates legitimate, and 1 indicates DGA generated, whilst softmax outputs are probability values in the range of (0-1). *Sigmoid* is used in the binary classification, while *Softmax* is used in the multiclass classification.

The training approach *Adam* neural network optimization is applied to adapt gradient optimization (per-parameter learning rate), and root mean square propagation (per-parameter learning rates are adapted based on the average of recent magnitudes of the weights). It adapts the learning rate using the second moment of the gradient (i.e. variance) by calculating the exponential moving average of the gradient and squared gradient. The hyperparameters of the models and their evaluations are discussed in Section VII-C.

V. DATA SET DESCRIPTION AND PREPROCESSING OF DATA SETS FOR VALIDATING THE PROPOSED SYSTEM

The proposed botnet detection system was evaluated on two data sets for discovering botnets from DNS traffic. The first data set is used for similarity checker, and the second one is used for DGA detection. The similarity checker data set is called *DS1*, and the DGA detection data set is called *DS2*. The *DS1* data set contains data of domain name similarity checker (*DS1 – D1*) and process name similarity checker (*DS1 – P1*). Both data sets have two classes, named Similar and Dissimilar. *DS1 – D1* and *DS1 – P1* are divided into three data sets, namely, training, validation, and testing having 40 thousand, 20 thousand and 20 thousand samples, respectively. Table V shows the number of samples in each split for both the *DS1 – D1* and *DS1 – P1* data sets.

The *DS1 – P1* data set is formed using the National Software Reference Library (NSRL)³. This contains the .exe and .dll files. The file names contain at least four characters, and legitimate pairs are created by estimating an all-to-all edit distance. The following distance function is used to retain the process name pairs

$$d(x1, x2) \leq 3 \quad (4)$$

Where d denotes the edit distance function. The Levenshtein distance function is most often used. A pair of malicious process names are created by conducting spoofing attacks that utilized the name of files extracted from NSRL. Thousands of character swaps are made using both Unicode and ASCII characters for spoofing attacks. The data set *DS1 – P1* and *DS1 – D1* are similar, and 100,000 active web domains list is used in order to generate data set *DS1 – P1*. The constraint on edit distance ($d \leq 3$) was removed while preparing the data set. This is due to the inadequate non-spoofing pairs with a distance of less than four. The legitimate strings in both *DS1 – P1* and *DS1 – D1* data sets are composed of ASCII characters only.

The *DS2* data set called AmritaDGA was used as part of the DMD 2018 shared task⁴ for detecting malicious domain

³<http://www.nsr1.nist.gov/>

⁴<https://nlp.amrita.edu/DMD2018/>

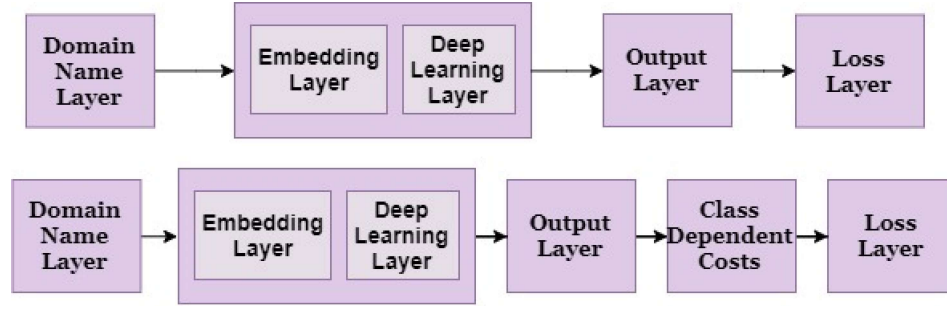


Fig. 8. Architecture of cost-insensitive and cost-sensitive deep learning.

TABLE IV
CHARACTER LEVEL DEEP LEARNING ARCHITECTURES.

Name	Architecture	Task
Endgame [40]	LSTM	Detecting and categorizing domain names that are generated by DGAs
Invincea [59]	CNN	To detect malicious URLs, file paths and registry keys
CMU [60]	Bidirectional recurrent structures	Social media text classification, Twitter
MIT [61]	Hybrid of CNN and LSTM	Social media text classification, Twitter
NYU [62]	Stacked CNN layers	Text classification

TABLE V
DS1 – D1 AND DS1 – P1 SPOOFING DATA SET DETAILS.

Data set	Data split	Number of samples	
		Similar	Dissimilar
DS1-D1 Domain names	Training set	14,279	25,721
	Validation set	7,149	12,851
	Testing set	7,090	12,910
Total		28,518	51,482
DS1-P1 Process names	Training set	15,123	24,877
	Validation set	14,827	5,173
	Testing set	8,431	11,569
Total		38,381	41,619

names. The shared task contains two subtasks, whereby task 1 classifies the domain name, either DGA generated or legitimate, and task 2 categorizes the DGA generated domain names to the corresponding categories along with identifying the domain name as DGA. We name the data set of task 1 'DS2 – V1' and the data set of task 2 'DS2 – V2'. DS2 – V1 data has two types, namely, Legitimate and DGA generated, which are divided into Training, Testing 1, and Testing 2. The Training data consists of 655,683 from legitimate, and 135,056 from DGA generated. The Testing 1 data consists of 2,349,331 from legitimate, and 108,076 from DGA generated. The Testing 2 data consists of 182 from legitimate, and 2,740 from DGA generated. DS2-V2 data has legitimate and DGA generated domain names. The DGA generated domain names are labeled with their DGA domain name family. The training, testing 1 and testing 2 of DS2-V2 contains 397,777 (legitimate: 100,000, DGA generated: 297,777), 587,112 (legitimate: 120,000, DGA generated: 467,112), 103,200 (legitimate:

40,000, DGA generated: 63,200) domain names respectively. The detailed statistics of each DGA malware family of the DS2-V2 data set is available in DMD-2018⁴. The detailed statistics of DS2-V1 and DS2-V2 shows that both the data sets are highly imbalanced and the cost-insensitive models are sensitive to these type of data sets. To handle this type of data, in this work we propose the cost-sensitive models. The detailed statistics for task 2 is reported in DMD-2018⁴. Each task contains two types of data, one is collected from publically available sources, and the other one is collected from a real-time environment.

VI. EVALUATION MEASURES

We adopt commonly used quality metrics to compare the correctness and accuracy of the classification of the DGA malicious detection model. We use the following terms for determining the quality of the classification models:

- True Positive (TP) - the number of data items correctly classified to the positive class.
- True Negative (TN) - the number of data items correctly classified to the negative class.
- False Positive (FP) - the number of data items wrongly classified to the positive class.
- False Negative (FN) - the number of data items wrongly classified to the negative class.
- **Accuracy** - is the measure that gives the total number of correct predictions made out of all the predictions made by the model. It is a good measure to use when target classes are nearly balanced in the data.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \quad (5)$$

- **Precision** - is the measure for the ability of the model to give quality positive predictions. It can be interpreted

as the probability that positive prediction made by the model is positive.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (6)$$

- **Recall** - is the ratio of the true positive (TP) divided by the total of true positive (TP) and false negative (FN) predictions.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (7)$$

- **F1-score** - is given by the harmonic mean between precision and recall measures. It can be a better measure when target classes are unevenly distributed.

$$\text{F1 - Score} = 2 \times \frac{(\text{Recall} \times \text{Precision})}{(\text{Recall} + \text{Precision})} \quad (8)$$

- **True Positive Rate (TPR)** - True positive rate (TPR) is a quality metric that reflects the ability of the DGA malicious detection model to detect malicious domain names using the deep learning classifiers. TPR is the ratio between true positive values and the total true positive and false negative values.
- **False Positive Rate (FPR)** - False positive rate (FPR) is a quality metric that reflects the frequency exhibited by the DGA malicious detection model in making a mistake to identify non-malicious domain names as malicious using the deep learning classifiers. FPR is the ratio between false positive values and the total false positive and true negative values.

The ROC curve plots two parameters, TPR, and FPR showing the performance of a classification model at all classification thresholds. It signifies the diagnostic ability of the binary classifiers and it shows the ability of the classifier in distinguishing between the classes. It is plotted by the predicted probabilities of the algorithm, namely TPR and FPR. Generally, ROC is a probability curve and Area Under Curve (AUC) that is a degree or measure of separability. AUC is defined as;

$$\text{AUC} = \int_0^1 \frac{TP}{TP + FN} d \frac{FP}{TN + FP} \quad (9)$$

$$\text{TPR} = \frac{TP}{TP + FP} \quad (10)$$

$$\text{FPR} = \frac{FP}{FP + TN} \quad (11)$$

VII. EXPERIMENTS, RESULTS, AND OBSERVATIONS

The machine learning algorithms were implemented using Scikit-learn⁵, and deep learning architectures were implemented using TensorFlow⁶ and Keras⁷. GPU enabled machines were used in all experiments. The GPU was NVidia GK110BGL Tesla K40 and CPU had a configuration (32

GB RAM, 2 TB hard disk, Intel(R) Xeon(R) CPU E3-1220 v3 @ 3.10GHz) running over 1 Gbps Ethernet network. All the source codes are publically made available for further research⁸.

A. Discussions of Data visualization for DNS features

To accurately understand the characteristics of DNS data, scatter plot, histogram, and density plot were used. These plots help to identify the correlation among various features. We used the following three well-known statistical features to identify the influences on DGA domain detection:

- Domain name length is estimated based on the length of domain name. We drop the domain names that are lesser than five. The shorter domain names can be handled using DNS blacklists. The high domain name length can also be an indicator for DGA domain detection.
- Domain name entropy computes the entropy of character distribution and measures the randomness of each domain name. Entropy (more specifically, Shannon entropy) is an important concept in information theory and this concept is being used in DGA detection. Generally, the DGA domain names are gibberish combinations of letters and numbers. The higher entropy of a domain name is more likely to be malicious than linguistically coherent domains. The entropy (randomness) is defined as follows

$$H(X) = \sum_{i=1}^n P(x_i) I(x_i) = - \sum_{i=1}^n P(x_i) \log_b P(x_i) \quad (12)$$

Where H is the symbol for entropy, and X is the domain name input data, so $H(X)$ means The entropy of data X . $P(x_i)$ is defined as follows:

$$P(x_i) = \frac{\text{freq}(x_i)}{\sum_j \text{freq}(x_j)} \quad (13)$$

Where $\text{freq}(x_i)$ is the character count of a domain name and $\sum$ denotes the summation.

- Domain name n-gram is estimated based on the sequence of characters of a domain name. Here 'n' can be any number and in this work, the n-gram is chosen in a range of (3-5), where the minimum document frequency set by 0.0001, and the maximum document frequency set by 1.0. Generally, the DGA domain name is generated randomly and this can contain different distribution of characters than those generated based on natural language. This difference helps to differentiate between the legitimate and DGA generated domain name.

Table VI shows some sample domain names with entropy and length features. From the table, it can be seen that the legitimate domain name 'google' has less entropy when compared to the other DGA generated domain names. However, this may not be true in all the test cases. It is easy to detect DGA domain names whose length is long. But there are domain names generated by DGA malware in recent days which are lesser in length. Thus, length can be used to detect DGA generated domain names to a limited stage.

⁸ <https://github.com/vinayakumarr/IoT-Botnet>

⁵<https://scikit-learn.org/stable/>

⁶<https://www.tensorflow.org/>

⁷<https://keras.io/>

TABLE VI
SAMPLE DOMAIN NAME WITH ENTROPY AND LENGTH FEATURE

Domain	Type	Entropy	Length
google	legitimate	1.918	6
ummvzhi	DGA	2.522	7
ungszh	DGA	2.585	6
pfvfhkinnb9z4x	DGA	3.64	15
goo-net	legitimate	2.522	7

Further, n-gram is calculated for domains from Table VI and it is listed in Table VII. The n-gram uses ngram_range=(3,5) and the minimum domain name and maximum frequency as 0.0001 and 1.0 respectively.

TABLE VII
SAMPLE OF N-GRAM FOR A LIST OF DOMAIN NAME USED IN TABLE VI

n-gram	Count
goo	0.301
-ne	0.0
-net	0.0
9z4	0.0
9z4x	0.0
b9z	0.0
b9z4	0.0
b9z4x	0.0
fhr	0.0
fhrk	0.0

It can be observed from Table VII that except 'goo', all terms count are 0. The term 'goo' appeared in legitimate domain names. Generally, n-gram can serve as a good feature to extract the similarity of the top-ranked Alexa domain names.

The scatter plot is drawn based on the domain name length and domain name entropy, which is shown in Figure 13 for $DS1 - P1$, Figure 9 for $DS1 - D1$ and Figure 17 for $DS2 - V1$.

We also represented the scatter plots for $DS1 - P1$, $DS1 - D1$, $DS2 - V1$ data sets, which use the domain name length in the X-axis and similar n-gram matches in the Y-axis, as shown in Figure 14 for $DS1 - P1$, Figure 10 for $DS1 - D1$ and Figure 18 for $DS2 - V1$. Likewise, the scatter plot is drawn for $DS1 - P1$, $DS1 - D1$, and $DS2 - V1$ data sets using the domain name entropy on the X-axis and similar n-gram matches on the Y-axis, as depicted in Figure 15 for $DS1 - P1$, Figure 11 for $DS1 - D1$ and Figure 19 for $DS2 - V1$.

The domain name length, domain name entropy, and n-gram are not efficient features for the data set $DS1 - P1$ and $DS1 - D1$. These features are closely related and the Figures 9, 10, 11, 13, 14, and 15, shows clearly overlap between these features. This is primarily due to the reason that the domain names and process names are similar. However, for the data set 3, Domain name length, Domain name entropy, and n-gram can be used as features. Finally, we have drawn a density plot for all data sets and shown the density plot for $DS1 - P1$ in

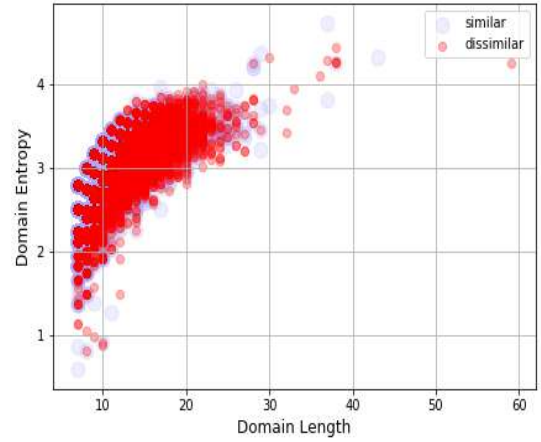


Fig. 9. $DS1 - D1$ data visualization based on domain name length and domain name entropy with scatter-plot.

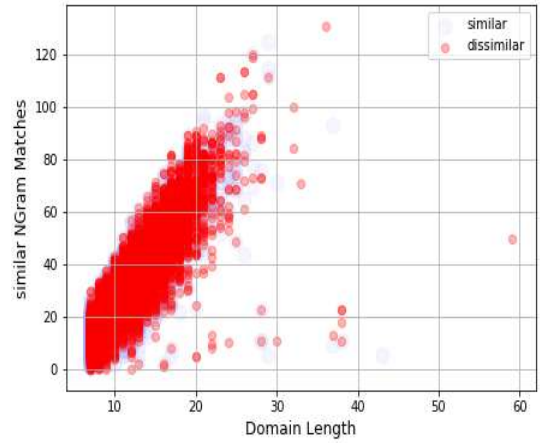


Fig. 10. $DS1 - D1$ data visualization based on domain name length and domain name n-gram with scatter-plot.

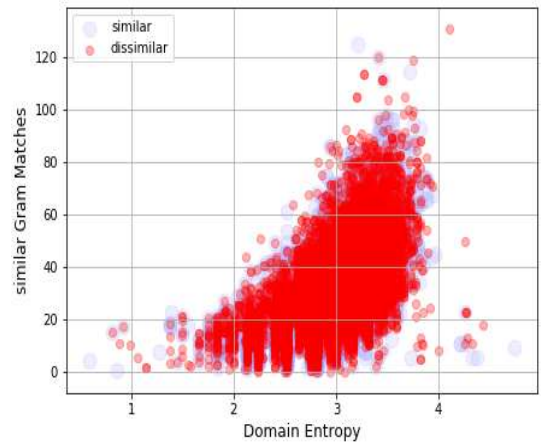


Fig. 11. $DS1 - D1$ data visualization based on domain name entropy and domain name n-gram with scatter-plot.

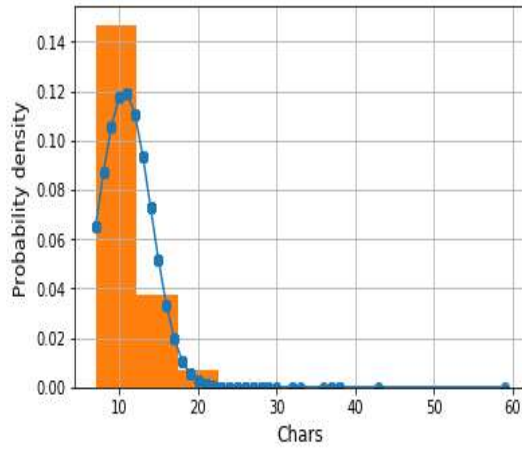


Fig. 12. $DS1 - D1$ data visualization based on domain name length.

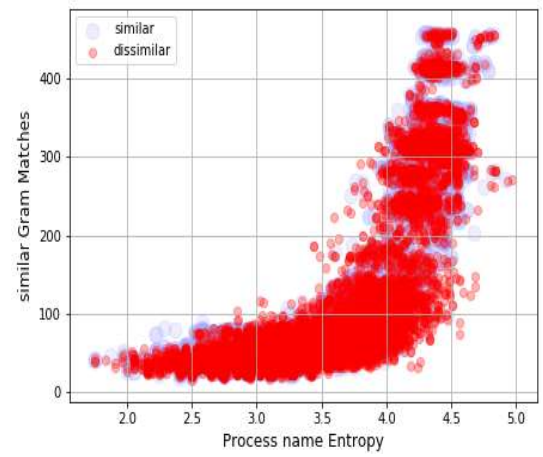


Fig. 15. $DS1 - P1$ data visualization based on process name entropy and domain name n-gram with scatter-plot.

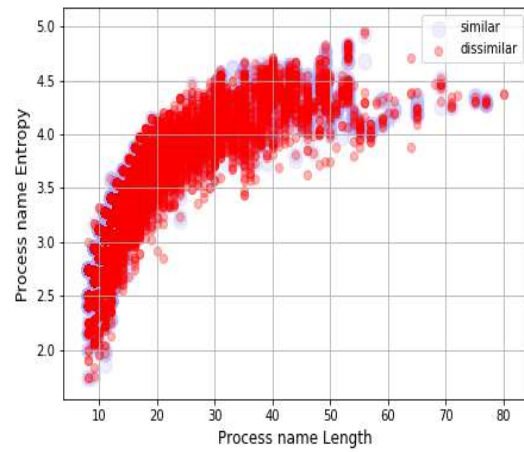


Fig. 13. $DS1 - P1$ data visualization based on process name length and process name entropy with scatter-plot.

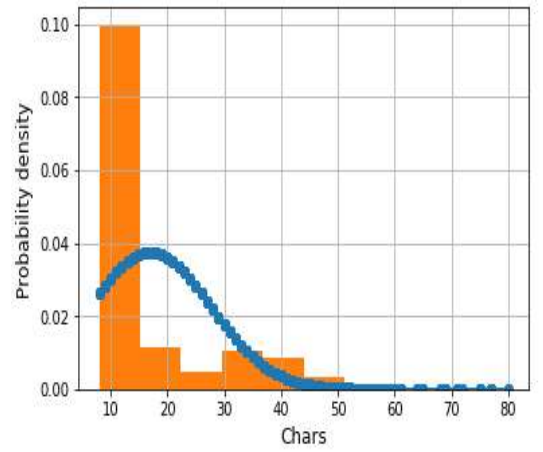


Fig. 16. $DS1 - P1$ data visualization based on domain name length.

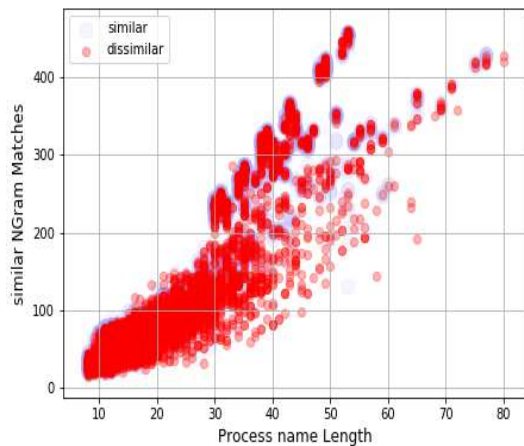


Fig. 14. $DS1 - P1$ data visualization based on process name length and domain name n-gram with scatter-plot.

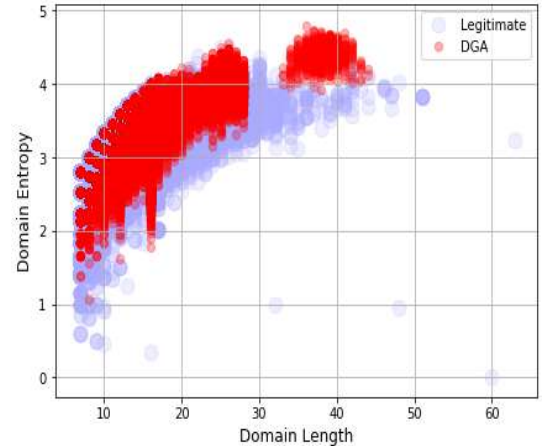


Fig. 17. $DS2 - V1$ data visualization based on domain name length and domain name entropy with scatter-plot

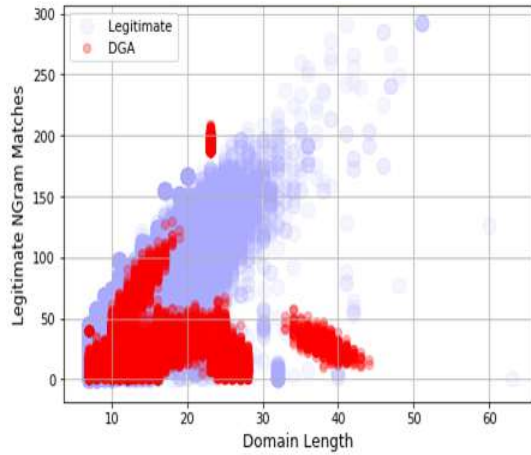


Fig. 18. $DS2 - V1$ data visualization based on domain name length and domain name n-gram with scatter-plot.

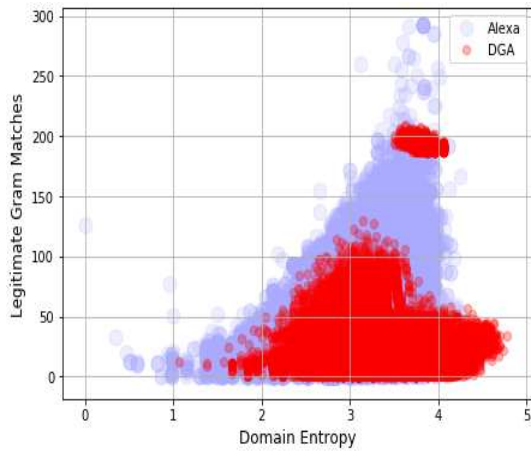


Fig. 19. $DS2 - V1$ data visualization based on domain name entropy, and domain name n-gram with scatter-plot.

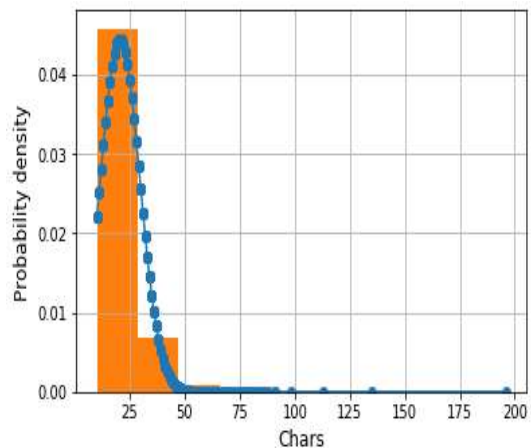


Fig. 20. $DS2 - V1$ data visualization based on domain name length.

Figure 16, $DS1 - D1$ in Figure 12 and $DS2 - V1$ in Figure 20. The density plot is used to find out the maximum length of the domain name. As shown in Figure 12, even though maximum length goes till 60, 23 can be taken because most of the domain names have a length of 23. Similarly, in Figure 20, the maximum length of the domain name can be set into 50 and Figure 16, the maximum length of the domain name can be set into 28. Thus by using these appropriate plots values for maximum length can be estimated. Identifying optimal value for the length of the domain name is an essential concept because it reduces the complexity of the character level domain name representation. However, in this work, we set the value for maximum length by finding the domain name which has the maximum length.

B. Discussions of Similarity checker

We used the domain name and process name data set from [36]. Initially, the data set is randomly divided into train, validation, and test data sets. Using the training data set, different siamese networks are trained to monitor the training accuracy across different epochs. The validation data set is used to avoid overfitting during the training phase. A dictionary size for the domain name is 37 and the process name is 82. Each character of a domain and process name is transformed into a numeric vector representation using the generated dictionary. The maximum length of the domain name is 100, and the process name is 85. The embedding size is set to 128 and also increasing the embedding size more than 128 does not provide better performance but introduces additional computational complexity. This converts the domain name vector representation into a dense vector representation. This vector representation is passed into deep learning layers, which extracts optimal features.

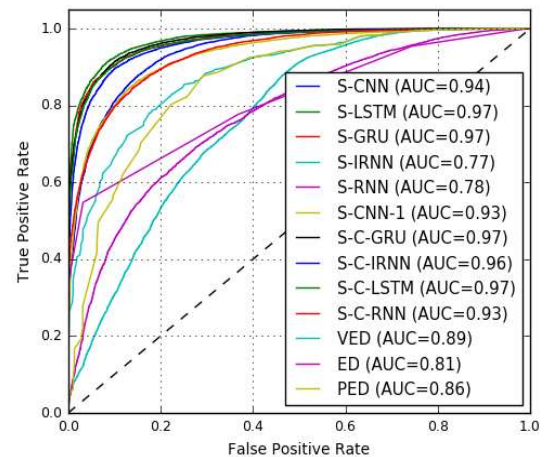


Fig. 21. ROC Curve for domain name data set $DS1 - D1$

The models are evaluated using the testing data set. The performance in terms of ROC for the domain name spoofing and process name spoofing is shown in Figures 21 and 22, respectively. The parameter details of all the different siamese networks are reported in Table VIII. The performance obtained by proposed methods performed well in compared to the existing siamese networks [36] and methods based on conventional

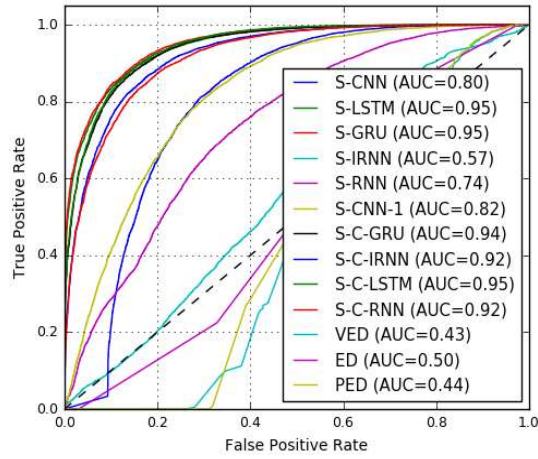


Fig. 22. ROC Curve for domain name data set $DS1 - P1$.

edit distance and visual edit distance [38], [37]. The proposed method contains less number of parameters compared to the existing method based on siamese networks [77]. Thus the proposed method is computationally inexpensive compared to the existing siamese networks.

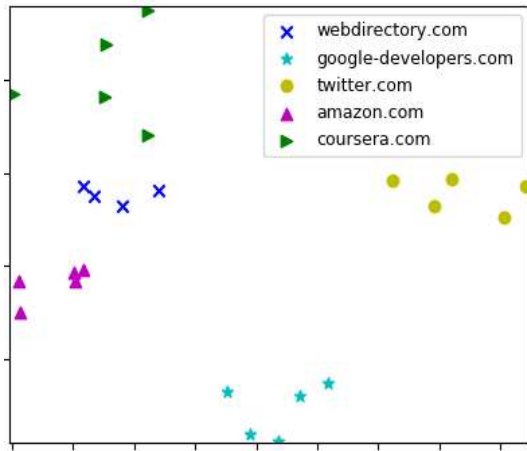


Fig. 23. Feature vector visualization using t-SNE - $DS1 - D1$ domain name spoofing data set.

The performances obtained by edit distance methods in terms of ROC-AUC close to 0.5 for process name data set. This is mainly because the samples of spoofing and non-spoofing pairs contain edit distance lesser than 3. Additionally, the performance obtained by visual edit distance is comparatively less compared to a classical edit distance method. This indicates that manually estimating the distance measure is often tricky. The data set includes only ASCII characters and supposes if the Unicode characters are contained, the possible number of unique characters is extremely large. Manually extracting all to all distances from each character is inconvenient. There may be a possibility to learn an all to all distance between characters, but creating such a kind of data set manually is a tedious task, and it is expensive. The performances in terms of ROC-AUC obtained by edit distance methods are reasonable compared to the process name data set

due to the edit distance of non-spoofing pairs of the domain name is greater than 3.

TABLE VIII
PARAMETER DETAILS OF SIAMESE NETWORKS FOR $DS1 - D1$ AND $DS1 - P1$ SPOOFING DATA SET.

Model	Domain Name Spoofing	Process Name Spoofing
	Total Parameters	Total Parameters
S-CNN [36]	148,832	148,832
S-RNN ([77])	58,496	71,296
S-IRNN ([77])	58,496	71,296
S-LSTM ([77])	157,184	169,984
S-GRU ([77])	124,288	137,088
S-CNN-1 (Proposed Method)	74,944	87,744
S-C-RNN (Proposed Method)	76,074	88,874
S-C-IRNN (Proposed Method)	76,074	88,874
S-C-LSTM (Proposed Method)	104,424	117,224
S-C-GRU (Proposed Method)	94,974	107,774

The best siamese network pairs of domain names are shown in Figure 23. This is obtained using t-SNE. Primarily, PCA projection is applied to the high dimensional feature vectors which reduce the number of dimensions to two. Each pair contains 1 legitimate domain name and 4 homoglyphs, a shown in Table IX. The domain name consists of webdirectory.com, google-developers.com, twitter.com, amazon.com, and coursera.com along with four homoglyph attacks for each domain. Note that each pair of domain names has completely appeared in a separate cluster. Each cluster is tightly associated with the legitimate domain name and four homoglyph attacks. This type of discrimination allows us to predict spoofing attacks with very low false positive rates.

C. Discussions of DGA detection based deep learning

As discussed, initially a dictionary is constructed and each character of a domain name is converted into numeric by looking at the dictionary. The dictionary size for the $DS2$ data set is 39. The valid character set is shown below:

-, ., 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, , a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z

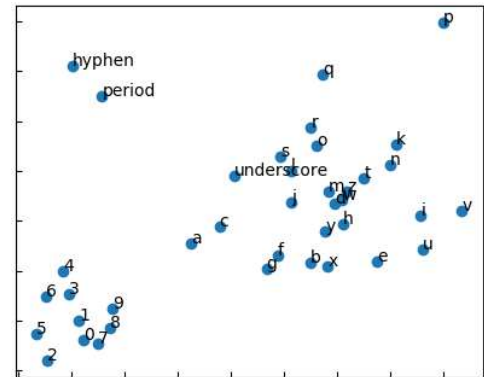


Fig. 24. Character level embedded feature vectors learned by the model are represented using two-dimensional linear projection (PCA) with t-SNE. Note that models groups feature vectors based on the similarity.

These domain name vectors are transformed into the same length by using the maximum size. The maximum length

TABLE IX

THE LEGITIMATE DOMAIN NAMES ARE PLACED IN THE FIRST ROW AND THE SPOOFED DOMAIN NAME OF THE CORRESPONDING LEGITIMATE DOMAIN NAMES ARE PLACED IN OTHER FOUR ROWS - $DS1 - D1$ DATA SET.

Domain name					Type
webdirectory.com	google-developers.com	twitter.com	amazon.com	coursera.com	Legitimate
webirectory.com	google-developebs.com	twitteb.com	amazon_.com	coursesu.com	Homoglyph
webdhrectopy.com	googlg-deveiopers.com	tvltter.com	amazon-n.com	cortera.com	Homoglyph
vebdirmctopy.com	goo_gle-_deveiopers.com	tvivtre.com	imzaonn.com	coursat.com	Homoglyph
webtrectry.com	google-deveiopers_.com	[twitter.com	amazōn.com	coursehero.com	Homoglyph

for $DS2$ Training is 91; Testing 1 is 82, and Testing 2 is 82. Then, the domain name vectors are reduced into of size maximum length by discarding the vectors which are more than maximum length and padding 0's to the domain name vectors if the vectors are less than the maximum length. These vectors are passed into Keras character level embedding. This has three parameters, such as embedding size, the maximum length of the input domain name and dictionary size. The embedding size is set to 128 and also increasing the embedding size more than 128 does not provide better performance but introduces additional computational complexity. This layer generally learns the high-level feature representation for each character of a domain name.

The high dimensional feature vectors of the embedding layer are passed into t-SNE. This turns the high dimensional feature vector into two dimensional, shown in Figure 24. Almost the numbers, characters, and special characters have completely appeared in a separate cluster. This indicates that the embedding layer has learned the complete separation among these three different groups. This type of discrimination allows the deep learning model to learn the sequence, syntactic and semantic similarity among characters and achieve a very low false positive rate to distinguish between the DGA generated and legitimate domain names. The Keras character level embedding layer follows various deep learning layers that learn the optimal feature representation and finally, optimal features are passed into a fully connected layer for classification. In between the deep learning layer and fully connected layer, we embed the dropout layer, which helps to prevent model overfitting by setting a random subset of neurons to zero.

The hyperparameters of the deep learning models are adopted as follows:

- 1) Cost items: We found that introducing the cost items in the multiclass imbalanced DGA problem increases the performance compared to the cost-sensitive deep learning models.
- 2) The dimension of embedding size: The dimension of embedding size is a hyperparameter, and we experimented with sizes 32, 64, 128 and 256. We found that the best results were obtained with 128 dimensions.
- 3) Bidirectional: The performance slightly improved when the previous and future layers are included. It is because the inclusion doubles the dimensions of the output vector. When compared with unidirectional networks with the same total number of dimensions, the bidirectional

TABLE X

DMD-2018 AND PROPOSED METHOD TEST RESULTS ON AMRITADGA TESTING 1 DATA SET OF $DS1 - V1$.

Model	Accuracy	Precision	Recall	F1-score
RNN [53]	0.979	0.688	0.944	0.796
LSTM [53]	0.988	0.797	0.960	0.871
GRU [53]	0.987	0.791	0.946	0.861
CNN [53]	0.978	0.673	0.965	0.793
CNN-LSTM [53]	0.985	0.772	0.938	0.847
UWT [53]	0.99	0.966	0.828	0.89
Deep_Dragons [53]	0.988	0.96	0.807	0.877
CHNMLRG [53]	0.988	0.944	0.819	0.88
BENHA[53]	0.963	0.199	0.795	0.32
BharathibSSNCSE [53]	0.615	0.311	0.037	0.07
UniPI [53]	0.981	0.919	0.724	0.81
Josan [53]	0.989	0.947	0.822	0.88
DeepDGA Net [53]	0.976	0.938	0.658	0.773
FANCI [53]	0.992	0.849	0.992	0.915
Endgame [40]	0.992	0.852	0.992	0.917
Invincea [59]	0.992	0.849	0.992	0.915
CMU [60]	0.992	0.852	0.992	0.917
MIT [61]	0.992	0.851	0.992	0.916
NYU [62]	0.992	0.851	0.992	0.916
C-Endgame (Proposed)	0.992	0.852	0.992	0.917
C-Invincea (Proposed)	0.992	0.851	0.992	0.916
C-CMU (Proposed)	0.992	0.854	0.992	0.918
C-MIT (Proposed)	0.992	0.853	0.992	0.917
C-NYU (Proposed)	0.992	0.850	0.992	0.915

networks are slower and do not perform better.

- 4) The linear combination of inputs followed by non-linear activation methods constitute a fully connected layer. The addition of a fully connected layer between the CNN and output layer may enhance the model performance by applying a non-linear transformation to the state vector capturing the character sequence that makes up the domain name. Nevertheless, the size of the parameter space is increased by this type of architecture: if the size of the dense layer is M and the size of the state vector is N , an additional $(M + 1) * N$ parameters should be learned.
- 5) Usage of additional hidden layer in recurrent structures: There is no improvement in the performance when a dense layer is included between the last recurrent unit and the output layer.

Initially, using the train data set of $DS2$, we trained all

TABLE XI
DMD-2018 AND PROPOSED METHOD TEST RESULTS ON AMRITADGA
TESTING 2 DATA SET OF $DS1 - V1$.

Model	Accuracy	Precision	Recall	F1-score
RNN [53]	0.767	1.000	0.752	0.858
LSTM [53]	0.700	0.999	0.680	0.809
GRU [53]	0.718	0.999	0.700	0.823
CNN [53]	0.759	0.999	0.744	0.823
CNN-LSTM [53]	0.727	0.999	0.709	0.829
UWT [53]	0.766	0.751	0.999	0.86
Deep_Dragons [53]	0.783	0.77	0.999	0.87
CHNMLRG [53]	0.787	0.999	0.774	0.87
BENHA [53]	0.564	0.55	0.974	0.7
BharathibSSNCSE [53]	0.562	0.559	0.956	0.71
UniPI [53]	0.714	0.696	0.999	0.82
Josan [53]	0.711	0.692	0.999	0.82
DeepDGA Net [53]	0.782	0.769	0.997	0.869
FANCI [53]	0.791	0.999	0.777	0.874
Endgame [40]	0.807	0.999	0.795	0.885
Invincea [59]	0.796	0.999	0.783	0.878
CMU [60]	0.820	0.999	0.809	0.894
MIT [61]	0.815	0.999	0.803	0.891
NYU [62]	0.801	0.999	0.789	0.882
C-Endgame (Proposed)	0.838	0.999	0.828	0.906
C-Invincea (Proposed)	0.825	0.999	0.815	0.897
C-CMU (Proposed)	0.845	0.999	0.835	0.910
C-MIT (Proposed)	0.841	0.999	0.831	0.907
C-NYU (Proposed)	0.832	0.999	0.822	0.902

the various character-based, cost-sensitive, and cost-insensitive models. The cost-sensitive models are represented in the tables by prefix 'C' (e.g. C-CMU). These models were tested on two different types of data sets, Testing 1 and Testing 2. The detailed results are reported in Table X for Testing 1 and Table XI for Testing 2. The proposed models performed well compared to all the existing models in terms of F1-score, Precision, and Recall. More importantly, the cost-sensitive model achieved better performance with low false-positive rates. The performance obtained by all these models are closer, and better performance can be obtained by following an optimal parameter tuning method. Overall, the C-CMU model achieved better performance in both Testing 1 and Testing 2 data sets. Finally, these models are evaluated for DGA classification. The results are demonstrated in Table XII for Testing 1 and Table XIII for Testing 2 data sets. These models have achieved better accuracy compared to the existing models.

VIII. DISCUSSION

Generally, deep learning models are computationally expensive and slow during the training stage but in the testing stage, they are very fast and they can satisfy the time-critical requirement in the real-time processing botnet detection system. This paper presents a feature-less, real-time technique using deep learning, which operates on raw domain names (e.g., google.com, facebook.com, etc.). Whenever a new DGA appears which does not belong to any existing families, then the classifier can be retrained without the tedious process of

TABLE XII
DMD-2018 AND PROPOSED METHOD TEST RESULTS ON AMRITADGA
TESTING 1 DATA SET OF $DS2 - V2$.

Model	Accuracy	Precision	Recall	F1-score
RNN [53]	0.658	0.636	0.658	0.626
LSTM [53]	0.672	0.663	0.672	0.622
GRU [53]	0.649	0.655	0.649	0.601
CNN [53]	0.604	0.629	0.604	0.568
CNN-LSTM [53]	0.599	0.615	0.599	0.556
UWT [53]	0.633	0.618	0.633	0.602
Deep_Dragons [53]	0.683	0.683	0.683	0.64
CHNMLRG[53]	0.648	0.662	0.648	0.6
BENHA [53]	0.272	0.194	0.272	0.168
BharathibSSNCSE [53]	0.18	0.092	0.18	0.102
UniPI [53]	0.655	0.647	0.655	0.615
Josan [53]	0.697	0.689	0.697	0.658
DeepDGA Net [53]	0.601	0.938	0.601	0.576
CMU [60]	0.712	0.697	0.671	0.684
C-CMU (Proposed)	0.731	0.728	0.701	0.714

TABLE XIII
DMD-2018 AND PROPOSED METHOD TEST RESULTS ON AMRITADGA
TESTING 2 DATA SET OF $DS2V2$.

Model	Accuracy	Precision	Recall	F1-score
RNN [53]	0.662	0.627	0.662	0.609
LSTM [53]	0.669	0.695	0.669	0.627
GRU [53]	0.665	0.718	0.665	0.637
CNN [53]	0.643	0.691	0.643	0.596
CNN-LSTM [53]	0.658	0.676	0.658	0.625
UWT [53]	0.887	0.924	0.887	0.901
Deep_Dragons [53]	0.672	0.896	0.672	0.713
CHNMLRG [53]	0.674	0.683	0.674	0.648
BENHA [53]	0.429	0.34	0.429	0.272
BharathibSSNCSE [53]	0.335	0.229	0.335	0.223
UniPI [53]	0.671	0.641	0.671	0.619
Josan [53]	0.679	0.694	0.679	0.636
DeepDGA Net [53]	0.531	0.653	0.531	0.541
CMU [60]	0.891	0.931	0.901	0.915
C-CMU (Proposed)	0.899	0.934	0.905	0.919

manual feature engineering. Deep learning models work as a black box making it very difficult for the adversaries to reverse engineer and trick the model. Secondly, the presented technique has a significantly better true positive rate and false positive rate when compared with previously published retrospective and real-time approaches. Thirdly, the proposed system not only provides a binary prediction of whether a domain is DGA or not, but it also can accurately fingerprint a unique DGA's structure. Lastly, the presented algorithm can perform classification in real-time using absolutely no contextual information. the classification of a domain takes 10ms on commodity hardware. The system is easy to implement and it can run on virtually any security environment. The source code of this system is publicly available in GitHub⁸.

In this work, all the data sets are in English and the model trained using this type of data set cannot be deployed on non-English speaking countries. There may be a possibility that these type of model can misclassify the legitimate domain name as DGA generated. To handle the non-English domain

names, we will find a better way to collect the legitimate domain names and the inner mechanisms involved in these legitimate domain names.

The proposed system uses siamese networks to represent two domain names and proceeds to use simple thresholding for calculating the similarity of these representations. An attacker can leverage the predefined threshold and avoid being detected by the similarity checker via finding domain names that create closer representations that have a lower distance than the threshold. One solution for future works maybe training a classifier simultaneously with the siamese Networks in an end-to-end manner.

IoT devices generate a lot of data and the IoT environment is dynamic which makes the classical static security mechanisms inadequate. The context-aware computing aids to interpret the data meaningfully by producing context information and also adds a new dimension to the old fashion security by using context information to provide security decisions [63], [64], [65], [66].

In addition, IoT enabled smart city applications streaming data collection and its processing requires new approaches [67], [76]. In [68], authors propose IoT Streaming Data Integration (ISDI) framework which provides a complete streaming methodology for building rich vectors, describing important process characteristics (or features), suitable for classical or incremental machine learning algorithms. Data handling and preprocessing in IoT streaming is an important step in the IoT-enabled smart city applications, as we see many IoT sources and a central controller to manage them. The necessity of proper data preprocessing techniques for the needs of stream mining approaches still remains an open issue [69]. In [69], a general system for generating feature vector from diverse data streams is proposed which has to ability to improve the modeling abilities of real-world IoT systems. Apache Kafka, Splunk, and Flink are some of the streaming frameworks that can be used in real-time to handle the IoT streaming data in large-scale as future work.

IX. CONCLUSION AND FUTURE WORK

This paper has presented a botnet detection system based deep learning using DNS queries specific to IoT smart city applications. The proposed system is highly scalable on a commodity hardware server. The system includes a two-level framework that can handle big data in real-time processing. The proposed framework uses siamese networks to find out the similarity score at the first level and pass the dissimilar domain names into the second level. The second level uses deep learning architecture to detect and classify the domain name into their corresponding classes. The framework can be scaled out to big data of DNS features by adding additional computing resources to existing environments. The proposed botnet detection method based deep learning outperformed classical machine learning based systems. Since botnet generates domain names very frequently, the accurate resolution of the domain names can be very time-consuming. The proposed method can be deployed at an ISP level to monitor all the Internet-connected IoT devices.

We observed that for few DGA families, both the cost-sensitive and cost-insensitive models did not recognize them correctly. The complexity of DGA methods ranges from a simple method that uniformly generates domain names to the complicated ones that generate domain names in the distribution seen in real domains. for example, malware like 'ramnit' generates domains by performing various operations such as multiplication, division, and modulo on seed [70] whereas 'suppobox' malware generates domains by joining two random English words together [71]. Detecting malware like these is hard without using additional contextual information such as HTTP headers, passive DNS and aggregations of all NX-Domains throughout a network. A detailed analysis of these DGA malware families to further enhance the overall system performance remained as one of the significant directions towards future work.

ACKNOWLEDGEMENT

This work was supported by the Department of Corporate and Information Services, Northern Territory Government of Australia and in part by the Paramount Computer Systems, and in part by the Lakhshya Cyber Security Labs. We would like to thank NVIDIA, for the GPU hardware support to the research grant. We would also like to thank Computational Engineering and Networking (CEN) department for encouraging the research.

REFERENCES

- [1] McKinsay Global Institute, "Unlocking the Potential of the Internet of Things", June 2015, Source: <http://www.mckinsey.com/industries/hightech/our-insights/the-internet-of-things-the-value-of-digitizing-the-physical-world>.
- [2] Silva, S. S., Silva, R. M., Pinto, R. C., & Salles, R. M. (2013). Botnets: A survey. *Computer Networks*, 57(2), 378-403.
- [3] Meidan, Yair, Michael Bohadana, Yael Mathov, Yisroel Mirsky, Asaf Shabtai, Dominik Breitenbacher, and Yuval Elovici. "N-BaIoT Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders." *IEEE Pervasive Computing* 17, no. 3 (2018): 12-22.
- [4] Antonakakis, M., Perdisci, R., Nadji, Y., Vasiloglou, N., Abu-Nimeh, S., Lee, W., & Dagon, D. (2012, August). From Throw-Away Traffic to Bots: Detecting the Rise of DGA-Based Malware. In *USENIX security symposium* (Vol. 12).
- [5] Akamai, "State of the Internet Security Q3 2016", 2016. Source: <https://www.akamai.com/us/en/multimedia/documents/state-of-theinternet/q3-2016-state-of-the-internet-security-executive-summary.pdf>.
- [6] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *nature*, 521(7553), 436.
- [7] Koroniatis, Nickolaos, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. "Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset." *arXiv preprint arXiv:1811.00701* (2018).
- [8] Vinayakumar, R., Poornachandran, P., & Soman, K. P. (2018). Scalable Framework for Cyber Threat Situational Awareness Based on Domain Name Systems Data Analysis. In *Big Data in Engineering Applications* (pp. 113-142). Springer, Singapore.
- [9] Vinayakumar, R., Soman, K. P., & Poornachandran, P. (2018). Detecting malicious domain names using deep learning approaches at scale. *Journal of Intelligent & Fuzzy Systems*, 34(3), 1355-1367.
- [10] B. Krebs, "Who is Anna-Senpai, the Mirai Worm Author?", January 2017, Source: <https://krebsonsecurity.com/2017/01/who-is-anna-senpai-the-mirai-worm-author/>.
- [11] Anna-senpai, Mirai Source Code on GitHub, September 2016, Source: <https://github.com/jgambelin/Mirai-Source-Code>.
- [12] Akamai, "State of the Internet Security Q3 2016", 2016. Source: <https://www.akamai.com/us/en/multimedia/documents/state-of-theinternet/q3-2016-state-of-the-internet-security-executive-summary.pdf>.

- [13] B. Krebs, "KrebsOnSecurity hit with record DDoS," in KrebsOnSecurity, 2016. Source: <https://krebsonsecurity.com/2016/09/krebsonsecurity-hit-with-record-ddos/>.
- [14] B. Krebs, "Alleged vDOS Proprietors Arrested in Israel," in KrebsOnSecurity, 2016. Source: <https://krebsonsecurity.com/2016/09/alleged-vdosproprietors-arrested-in-israel/>.
- [15] B. Krebs, "The Democratization of Censorship," in KrebsOnSecurity, 2016. Source: <https://krebsonsecurity.com/2016/09/the-democratization-of-censorship/>.
- [16] R. Millman, "OVH suffers 1.1Tbps DDoS attack," in News, SC Magazine UK, 2016. Source: <http://www.scmagazineuk.com/ovh-suffers-11tbpsddos-attack/article/524826/>.
- [17] S. Hilton, "Dyn analysis summary of Friday October 21 Attack," in Dyn, 2016. Source: <http://hub.dyn.com/dyn-blog/dyn-analysis-summary-of-friday-october-21-attack>.
- [18] B. Krebs, "DDoS on Dyn impacts Twitter, Spotify, Reddit," in KrebsOnSecurity, 2016. Source: <https://krebsonsecurity.com/2016/10/ddos-ondyn-impacts-twitter-spotify-reddit/>.
- [19] Kenzo2017, "Eir P-660HW-T1 Vulnerability", 7th November 2016, Source: <https://devicereversing.wordpress.com/2016/11/07/eirs-d1000modem-is-wide-open-to-being>.
- [20] Uzair Amir, "Thousands of CCTV Devices Found DDoSing Small-Business Websites", June 2016, <http://www.securityweek.com/thousands-cctv-devices-abused-ddos-attacks>.
- [21] B. Krebs, "Did the Mirai Botnet really take Liberia Offline?," in KrebsOnSecurity, 2016. Source: <https://krebsonsecurity.com/2016/11/did-themirai-botnet-really-take-liberia-offline/>.
- [22] K. Beaumont, "Shadows Kill - Mirai DDoS botnet testing large scale attacks, sending threatening messages about UK and attacking researchers" in Medium, Medium, 2016. Source: <https://doublepulsar.com/@networksecurity>.
- [23] Janita, "DDoS Attack halts heating in Finland amidst winter," in Metropolitan, 2016. Source: <http://metropolitan.fi/entry/ddos-attackhalts-heating-in-finland-amidst-winter>.
- [24] M. Kumar, "DDoS Attack Takes Down Central Heating System Amidst Winter in Finland," in The Hacker News, The Hacker News, 2016. Source: <http://thehackernews.com/2016/11/heating-system-hacked.html>.
- [25] M. Beinart, "Russia: Hackers target financial sector," in Organized Crime and Corruption Reporting Project, 2016. Source: <https://www.occrp.org/en/daily/5790-russia-hackers-target-major-russian-banks>.
- [26] L. John, "Russian banks floored by withering DDoS attacks," in The Register, 2016. Source: <http://www.theregister.co.uk/2016/11/11/russianbanksddos/>.
- [27] A. Cuthbertson, "Russian Banks Become Latest Victim of Mirai Cyberattacks," in Newsweek, Newsweek Europe, 2016. Source: <http://www.newsweek.com/russian-banks-become-latest-victim-miraicyberattack-botnet-520>.
- [28] A. Nixon, J. Costello, and R. Tokazowski, "Flashpoint - flashpoint monitoring of Mirai shows attempted DDoS of trump and Clinton Websites," in Flashpoint Cybercrime, Flashpoint, 2016. Source: <https://www.flashpoint-intel.com/attempted-ddos-trump-and-clinton-websites/>.
- [29] A. Greenberg, "Hackers Target Pro-Clinton Phone Banks But Hit Trumps Too," in Wired, WIRED, 2016. Source: <https://www.wired.com/2016/11/hackers-target-pro-clinton-phonebanks-hit-trumps/>.
- [30] A. Uzunovic, "WikiLeaks Releases DNCLeak2; Suffers Massive DDoS Attack," in HackRead, HackRead, 2016. Source: <https://www.hackread.com/wikileaks-dncleak2-suffers-massive-ddosattack/>.
- [31] "DDoS attack on WikiLeaks stymies new #PodestaEmails release," in RT, RT International, 2016. Source: <https://www.rt.com/usa/365845-wikileaks-ddos-attack-assange/>.
- [32] C. Herberger, "As Cyber Security Programs Lose Their Moorings to Ransom-DoS: Radware Introduces the Ultimate Guide to Cyber Ransom", 2016. Source: <https://blog.radware.com/security/2016/09/radwareultimate-cyber-ransom-guide/>.
- [33] Deng, D., Li, G., & Feng, J. (2014). A pivotal prefix based filtering algorithm for string similarity search. In Proceedings of the 2014 ACM SIGMOD international conference on Management of data (pp. 673-684). ACM.
- [34] Deng, D., Li, G., Feng, J., & Li, W. S. (2013). Top-k string similarity search with edit-distance constraints. In Data Engineering (ICDE), 2013 IEEE 29th International Conference on (pp. 925-936). IEEE.
- [35] Wang, J., Li, G., & Fe, J. (2011, April). Fast-join: An efficient method for fuzzy token matching based string similarity join. In Data Engineering (ICDE), 2011 IEEE 27th International Conference on (pp. 458-469). IEEE.
- [36] Woodbridge, J., Anderson, H. S., Ahuja, A., & Grant, D. (2018, May). Detecting Homoglyph Attacks with a Siamese Neural Network. In 2018 IEEE Security and Privacy Workshops (SPW) (pp. 22-28). IEEE.
- [37] P. E. Black. (2008) Compute visual similarity of top-level domains. [Online]. Available: <https://hissa.nist.gov/black/GTLD/>.
- [38] Linari, A., Mitchell, F., Duce, D., & Morris, S. (2009). Typo-Squatting: The Curse of Popularity.
- [39] Bromley, J., Guyon, I., LeCun, Y., Sckinger, E., & Shah, R. (1994). Signature verification using a "siamese" time delay neural network. In Advances in neural information processing systems (pp. 737-744).
- [40] Woodbridge, J., Anderson, H. S., Ahuja, A., & Grant, D. (2016). Predicting domain generation algorithms with long short-term memory networks. arXiv preprint arXiv:1611.00791.
- [41] Lison, P., & Mavroeidis, V. (2017). Automatic Detection of Malware Generated Domains with Recurrent Neural Models. arXiv preprint arXiv:1709.07102.
- [42] Vinayakumar, R., Soman, K. P., Poornachandran, P., & Sachin Kumar, S. (2018). Evaluating deep learning approaches to characterize and classify the DGAs at scale. Journal of Intelligent & Fuzzy Systems, 34(3), 12651276.
- [43] Tran, D., Mac, H., Tong, V., Tran, H. A., & Nguyen, L. G. (2018). A LSTM based framework for handling multiclass imbalance in DGA botnet detection. Neurocomputing, 275, 2401-2413.
- [44] Yu, B., Pan, J., Hu, J., Nascimento, A., & De Cock, M. (2018). Character Level Based Detection of DGA Domain Names.
- [45] Mohan, V. S., Vinayakumar, R., Soman, K. P., & Poornachandran, P. (2018, May). Spoof net: Syntactic patterns for identification of ominous online factors. In 2018 IEEE Security and Privacy Workshops (SPW) (pp. 258-263). IEEE.
- [46] Vinayakumar, R., Soman, K. P., Poornachandran, P., Mohan, V. S., & Kumar, A. D. (2019). ScaleNet: Scalable and Hybrid Framework for Cyber Threat Situational Awareness Based on DNS, URL, and Email Data Analysis. Journal of Cyber Security and Mobility, 8(2), 189-240.
- [47] Curtin, R. R., Gardner, A. B., Grzonkowski, S., Kleymenov, A., & Mosquera, A. (2018). Detecting DGA domains with recurrent neural networks and side information. arXiv preprint arXiv:1810.02023.
- [48] Yu, B., Gray, D. L., Pan, J., De Cock, M., & Nascimento, A. C. (2017, November). Inline dga detection with deep networks. In Data Mining Workshops (ICDMW), 2017 IEEE International Conference on (pp. 683692). IEEE.
- [49] Feng, Z., Shuo, C., & Xiaochuan, W. (2017, December). Classification for DGA-Based Malicious Domain Names with Deep Learning Architectures. In 2017 Second International Conference on Applied Mathematics and information technology (p. 5).
- [50] Zago, M., Prez, M. G., & Prez, G. M. (2019). Scalable detection of botnets based on DGA. Soft Computing, 1-21.
- [51] Mac, H., Tran, D., Tong, V., Nguyen, L. G., & Tran, H. A. (2017, December). DGA Botnet Detection Using Supervised Learning Methods. In Proceedings of the Eighth International Symposium on Information and Communication Technology (pp. 211-218). ACM.
- [52] Anderson, H. S., Woodbridge, J., & Filar, B. (2016, October). DeepDGA: Adversarially-tuned domain generation and detection. In Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security (pp. 1321). ACM.
- [53] Vinayakumar R, Soman KP, Prabakaran Poornachandran, Mamoun Alazab, and Sabu M. Thampi, AmritaDGA: A Comprehensive Data set for Domain Generation Algorithms (DGAs). In Big Data Recommender Systems: Recent Trends and Advances, Institution of Engineering and Technology (IET)
- [54] Khrer, M., Rossow, C., & Holz, T. (2014, September). Paint it black: Evaluating the effectiveness of malware blacklists. In International Workshop on Recent Advances in Intrusion Detection (pp. 1-21). Springer, Cham.
- [55] Yadav, S., Reddy, A. K. K., Reddy, A. L., & Ranjan, S. (2010, November). Detecting algorithmically generated malicious domain names. In Proceedings of the 10th ACM SIGCOMM conference on Internet measurement (pp. 48-61). ACM.
- [56] Yadav, S., Reddy, A. K. K., Reddy, A. N., & Ranjan, S. (2012). Detecting algorithmically generated domain-flux attacks with DNS traffic analysis. IEEE/Acm Transactions on Networking, 20(5), 1663-1677.
- [57] Schppen, S., Teubert, D., Herrmann, P., & Meyer, U. (2018). FANCI: Feature-based Automated NXDomain Classification and Intelligence. In 27th USENIX Security Symposium (USENIX Security 18) (pp. 1165-1181).
- [58] Vinayakumar, R., Soman, K. P., Poornachandran, P., & Menon, P. A. deepdive on Machine learning for Cybersecurity use cases. Machine

Learning for Computer and Cyber Security: Principle, Algorithms, and Practices. CRC Press, USA.

- [59] J. Saxe and K. Berlin, eXpose: A character-level convolutional neural network with embeddings for detecting malicious urls, file paths and registry keys, arXiv preprint arXiv:1702.08568, 2017
- [60] B. Dhingra, Z. Zhou, D. Fitzpatrick, M. Muehl, and W. Cohen, Tweet2vec: Character-based distributed representations for social media, in Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, vol. 2, 2016, pp. 269274.
- [61] S. Vosoughi, P. Vijayaraghavan, and D. Roy, Tweet2vec: Learning tweet embeddings using character-level cnn-lstm encoder-decoder, in Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval, 2016, pp. 10411044.
- [62] X. Zhang, J. Zhao, and Y. LeCun, Character-level convolutional networks for text classification, in Advances in Neural Information Processing Systems, vol. 28, 2015, pp. 649657.
- [63] Perera, C., Zaslavsky, A., Christen, P., & Georgakopoulos, D. (2013). Context aware computing for the internet of things: A survey. IEEE communications surveys & tutorials, 16(1), 414-454.
- [64] Kayes, A. S. M., Rahayu, W., & Dillon, T. (2019). Critical situation management utilizing IoT-based data resources through dynamic contextual role modeling and activation. Computing, 101(7), 743-772.
- [65] Kayes, A. S. M., Rahayu, W., Dillon, T., Chang, E., & Han, J. (2019). Context-aware access control with imprecise context characterization for cloud-based data resources. Future Generation Computer Systems, 93, 237-255.
- [66] Kayes, A. S. M., Han, J., Rahayu, W., Dillon, T., Islam, M. S., & Colman, A. (2018). A policy model and framework for context-aware access control to information resources. The Computer Journal, 62(5), 670-705.
- [67] Kayes, A. S. M., Rahayu, W., Dillon, T., & Chang, E. (2018, August). Accessing data from multiple sources through context-aware access control. In 2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science And Engineering (Trust-Com/BigDataSE) (pp. 551-559). IEEE.
- [68] Doan Quang Tu, A. S. M. Kayes, Wenny Rahayu, Kinh Nguyen: ISDI: A New Window-Based Framework for Integrating IoT Streaming Data from Multiple Sources. AINA 2019: 498-511
- [69] Kenda, K., Kaži, B., Novak, E., & Mladen, D. (2019). Streaming Data Fusion for the Internet of Things. Sensors, 19(8), 1955.
- [70] Symantec, W32.Ramnit analysis. 2015-02-24, Version 1.0.
- [71] J. Geffner, End-to-end analysis of a domain generating algorithm malware family. Black Hat USA 2013, 2013.
- [72] Vinayakumar, R., Alazab, M., Jolfaei, A., Soman, K. P., & Poornachandran, P. (2019, May). Ransomware triage using deep learning: twitter as a case study. In 2019 Cybersecurity and Cyberforensics Conference (CCC) (pp. 67-73). IEEE.
- [73] Vinayakumar, R., Soman, K. P., Poornachandran, P., Alazab, M., & Jolfaei, A. (2019). DBD: Deep Learning DGA-Based Botnet Detection. In Deep Learning Applications for Cyber Security (pp. 127-149). Springer, Cham.
- [74] Ahmad, U., Song, H., Bilal, A., Alazab, M., & Jolfaei, A. (2019). Securing smart vehicles from relay attacks using machine learning. The Journal of Supercomputing, 1-18.
- [75] Kaur, M., Kaur, G., Sharma, P. K., Jolfaei, A., & Singh, D. (2019). Binary cuckoo search metaheuristic-based supercomputing framework for human behavior analysis in smart home. The Journal of Supercomputing, 1-24.
- [76] Zhang, H., Jolfaei, A., & Alazab, M. (2019). A Face Emotion Recognition Method Using Convolutional Neural Network and Image Edge Computing. IEEE Access.
- [77] Vinayakumar, R., & Soman, K. P. (2019). Siamese neural network architecture for homograph attacks detection. ICT Express.
- [78] Alazab, M. (2019). Deep Learning Applications for Cyber Security.
- [79] Alazab, M. (2015). Profiling and classifying the behavior of malicious codes. Journal of Systems and Software, 100, 91-102.
- [80] Huda, S., Abawajy, J., Alazab, M., Abdollahian, M., Islam, R., & Yearwood, J. (2016). Hybrids of support vector machine wrapper and filter based framework for malware detection. Future Generation Computer Systems, 55, 376-390.



R. Vinayakumar received the Ph.D. degree in computer science from Computational Engineering & Networking, Amrita School of Engineering, Coimbatore, Amrita Vishwa Vidyapeetham, India. He is currently a Postdoctoral research fellow in developing and implementing novel computational and machine learning algorithms and applications for big data integration and data mining with Cincinnati Children's Hospital Medical Center, Cincinnati, OH, USA. He has received MCA from Amrita Vishwa Vidyapeetham, Mysore in 2014 and BCA from JSS College of Arts, Commerce and Sciences, Ooty road, Mysore in 2011. He has several papers in Machine Learning applied to Cyber Security. His Ph.D. work centers on Application of Machine learning (some times Deep learning) for Cyber Security and discusses the importance of Natural language processing, Image processing and Big data analytics for Cyber Security. He has participated in several international shared tasks and organized a shared task on detecting malicious domain names (DMD 2018) as part of SSCC'18 and ICACCI'18. His research interests include machine learning and deep learning applications with natural language processing and image processing for Cyber Security. More details available at <https://vinayakumarr.github.io/>.



Mamoun Alazab received his PhD degree in Computer Science from the Federation University of Australia, School of Science, Information Technology and Engineering. He is an Associate Professor in the College of Engineering, IT and Environment at Charles Darwin University, Australia. He is a Cyber Security researcher and practitioner with industry and academic experience. Alazab's research is multi-disciplinary that focuses on Cyber Security and digital forensics of computer systems including current and emerging issues in the cyber environment like cyber-physical systems and the internet of things with a focus on cyber crime detection and prevention. He has more than 100 research papers. He delivered many invited and keynote speeches, 22 events in 2018 alone. He convened and chaired more than 50 conferences and workshops. He works closely with the government and industry on many projects. He is an editor on multiple editorial boards including Associate Editor of IEEE Access (2017 Impact Factor 3.5), Editor of the Security and Communication Networks Journal (2016 Impact Factor: 1.067) and Book Review Section Editor: Journal of Digital Forensics, Security and Law (JDFSL). He is a Senior Member of the IEEE.



S. Sriram is an M.Tech student in the Computational Engineering & Networking, Amrita School of Engineering, Coimbatore, Amrita Vishwa Vidyapeetham, India. He has received B.Tech in Computer Science and Engineering from Sastra University, Srinivasa Ramanujan Center, Kumbakonam, India in 2016. His areas of interest are Machine Learning, Deep Learning, Cyber Security, Computer vision, Image processing and the Internet of Things (IoT). More details available at <https://thecodinglama.github.io/>.



Quoc-Viet Pham (M'18) received the B.S. degree in electronics and telecommunications engineering from Hanoi University of Science and Technology, Vietnam, in 2013, and the M.S. and Ph.D. degrees, both in telecommunications engineering, from Inje University, South Korea, in 2015 and 2017 respectively. He is currently a research professor at Research Institute of Computer, Information and Communication, Pusan National University, South Korea. Prior to this position, he was with ICT Convergence Center, Changwon National University

and with High Safety Core Technology Research Center, Inje University, South Korea. He received the best PhD thesis award in Engineering from Inje University in 2017. His research interests include network optimization, mobile edge/cloud computing, and resource allocation for 5G wireless networks and beyond. He is a member of the IEEE.



K. P. Soman has 25 years of research and teaching experience at Amrita School of Engineering, Coimbatore. He has around 150 publications in national and international journals and conference proceedings. He has organized a series of workshops and summer schools in Advanced signal processing using wavelets, Kernel Methods for pattern classification, Deep learning, and Big-data Analytics for industry and academia. He authored books on "Insight into Wavelets", "Insight into Data mining", "Support Vector Machines and Other Kernel Methods" and "Signal and Image processing-the sparse way", published by Prentice Hall, New Delhi, and Elsevier. More details available at <https://nlp.amrita.edu/somankp/>.



K Simran is an M.Tech student in the Computational Engineering & Networking, Amrita School of Engineering, Coimbatore, Amrita Vishwa Vidyapeetham, India. She has received B.E in Information Technology from MVSR engineering college, Hyderabad, India in 2018. Her areas of interest are Machine Learning, Deep learning, Natural language processing, Cyber Security, Image processing and the Internet of Things (IoT). More details available at <https://simranketha.github.io/>.